

Question 1

An analyst can query tables in corp.sandbox but gets a permission error on **CREATE TABLE corp.sandbox.scratch AS SELECT**. She holds **USE CATALOG** on corp, **USE SCHEMA** on corp.sandbox, and **SELECT** on the source.

Which grant fixes this?

- A. **MODIFY** on the schema corp.sandbox, which lets her create new tables in that schema, together with the **USE** privileges that she already holds there
- B. **CREATE TABLE** on the schema corp.sandbox, which lets her create tables there, together with the **USE** privileges that she already holds
- C. **ALL PRIVILEGES** on the source table, which lets her create new tables from it in corp.sandbox, together with the **USE** privileges that she holds
- D. **CREATE SCHEMA** on the catalog corp, which lets her create new tables in corp.sandbox, together with the **USE** privileges that she already holds there

Correct answer: B

B. CREATE TABLE on the schema corp.sandbox, which lets her create tables there, together with the **USE** privileges that she already holds

Explanation

Correct answer:

CREATE TABLE on the schema corp.sandbox, which lets her create tables there, together with the **USE** privileges that she already holds

Why this is correct:

Creating a table in an existing Unity Catalog schema requires **CREATE TABLE** on that schema, alongside **USE CATALOG** and **USE SCHEMA** for namespace access. The analyst already holds both usage privileges and **SELECT** on the source, so granting **CREATE TABLE** on corp.sandbox is the missing minimum privilege.

Why the other options are wrong:

MODIFY on the schema corp.sandbox, which lets her create new tables in that schema, together with the USE privileges that she already holds there: **MODIFY** permits changing data in an existing table; it does not authorize creating a new table object in the schema. The needed privilege is **CREATE TABLE**.

CREATE SCHEMA on the catalog corp, which lets her create new tables in corp.sandbox, together with the USE privileges that she already holds there: **CREATE SCHEMA** allows creating schemas within the catalog, not tables inside an existing schema. It grants a different object-creation capability.

ALL PRIVILEGES on the source table, which lets her create new tables from it in corp.sandbox, together with the USE privileges that she holds: Privileges on the source table govern reading or modifying that source, not creating a destination table in another schema. The missing grant belongs on corp.sandbox.

References:

1. Unity Catalog privileges reference | Databricks on AWS:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/access-control/privileges-reference>

Question 2

A downstream job should start soon after a PostgreSQL table changes. The team reads that table through a Lakehouse Federation foreign catalog and has no other copy of the data in Unity Catalog.

Which way of starting the job fits?

- A. A file arrival trigger on the foreign catalog's storage path, since federation writes each PostgreSQL change as a new file under that storage path
- B. A time-based schedule that fits the freshness need, since foreign tables from federation are not among the types that table update triggers support
- C. A continuous trigger that reads the foreign table, since a continuous job starts a new run only after the foreign table in PostgreSQL has changed
- D. A table update trigger on the foreign table, since federation passes each PostgreSQL commit through to Unity Catalog as a new commit on that table

Correct answer: B

B. A time-based schedule that fits the freshness need, since foreign tables from federation are not among the types that table update triggers support

Explanation

Correct answer:

A time-based schedule that fits the freshness need, since foreign tables from federation are not among the types that table update triggers support

Why this is correct:

Table update triggers monitor supported Unity Catalog tables and views, but federated foreign tables are not a supported source in this scenario. Since PostgreSQL is queried in place and no Delta/UC-managed copy exists to monitor, a time-based schedule is the applicable way to meet the freshness target.

Why the other options are wrong:

A table update trigger on the foreign table, since federation passes each PostgreSQL commit through to Unity Catalog as a new commit on that table: Federation exposes remote PostgreSQL objects for querying; it does not convert each database commit into a Unity Catalog table commit. Foreign-table changes therefore do not fire this trigger.

A continuous trigger that reads the foreign table, since a continuous job starts a new run only after the foreign table in PostgreSQL has changed: This trigger restarts after a job run completes or fails; it does not subscribe to PostgreSQL table changes. It would run regardless of whether the source changed.

A file arrival trigger on the foreign catalog's storage path, since federation writes each PostgreSQL change as a new file under that storage path: Federation does not materialize PostgreSQL updates as files in a Databricks storage path. A file-arrival trigger has no such path to monitor.

References:

1. Trigger jobs when source tables are updated | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/jobs/trigger-table-update>

Question 3

A job's duration jumped from 20 to 35 minutes on one day and stayed there. Input volume and cluster size are flat across the period, and the job's bundle was redeployed from a new commit that morning.

What should the engineer do first?

- A. Add workers to the job cluster, as a lasting step change in duration shows that the job has outgrown the compute capacity that it currently has
- B. Compare task durations from the runs before and after that deployment, then review the commit's changes to the tasks that slowed down the most
- C. Wait for more runs to build a longer baseline, as a single step change in duration cannot be told apart from ordinary day-to-day variation
- D. Enable predictive optimization on the job's tables, as a lasting step change in duration points to small files building up in them over time

Correct answer: B

B. Compare task durations from the runs before and after that deployment, then review the commit's changes to the tasks that slowed down the most

Explanation

Correct answer:

Compare task durations from the runs before and after that deployment, then review the commit's changes to the tasks that slowed down the most

Why this is correct:

The duration jump coincides with a specific bundle deployment while input volume and cluster size remain stable. Comparing task-level durations before and after the deployment identifies which components regressed; reviewing the commit then connects the regression to code or configuration changes. This uses the strongest change signal before scaling compute or altering tables.

Why the other options are wrong:

Wait for more runs to build a longer baseline, as a single step change in duration cannot be told apart from ordinary day-to-day variation: A 75% lasting step increase aligned with a deployment is actionable evidence, not merely a noisy single-run fluctuation. Waiting delays investigation of the most likely cause.

Add workers to the job cluster, as a lasting step change in duration shows that the job has outgrown the compute capacity that it currently has: Adding workers changes capacity but does not explain why performance shifted on the deployment date while workload and size stayed flat. First isolate which tasks slowed and inspect the change.

Enable predictive optimization on the job's tables, as a lasting step change in duration points to small files building up in them over time: Predictive optimization addresses managed-table maintenance, but the facts give a specific code deployment and stable workload. There is no evidence that file accumulation caused the sudden shift.

References:

1. Diagnose Lakeflow Jobs performance | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/jobs/diagnose-job-performance>

Question 4

A job scheduled every 30 minutes sometimes runs for 45 minutes. A scheduled start arrives while the previous run is still active.

```
max_concurrent_runs: 1
queue:
  enabled: false
schedule:
  quartz_cron_expression: "0 0/30 * * * ?"
```

What happens to the new run?

- A.** It is skipped, because the job already has as many active runs as its limit allows and a new run has nowhere to wait for a free slot
- B.** It starts at once, because the scheduler raises the concurrency limit for the new run while the earlier run of the job is still active
- C.** It cancels the active run and then starts, because a newer scheduled run takes priority over an older run that belongs to the same job
- D.** It waits in the queue until the active run ends, because a scheduled run always queues whenever the concurrency limit has been reached

Correct answer: A

A. It is skipped, because the job already has as many active runs as its limit allows and a new run has nowhere to wait for a free slot

Explanation

Correct answer:

It is skipped, because the job already has as many active runs as its limit allows and a new run has nowhere to wait for a free slot

Why this is correct:

The job is already at its maximum of one active run, and queueing is explicitly disabled. A scheduled trigger that arrives during the active run has no queued slot to wait for, so that run is skipped rather than starting concurrently or canceling the existing run. Enabling queueing would change this outcome.

Why the other options are wrong:

It starts at once, because the scheduler raises the concurrency limit for the new run while the earlier run of the job is still active: The scheduler does not raise `max_concurrent_runs` automatically to accommodate a scheduled run. The configured limit remains one active run.

It waits in the queue until the active run ends, because a scheduled run always queues whenever

the concurrency limit has been reached: The queue is disabled in the configuration, so the new run cannot wait for the active run to finish. When concurrency is reached with no queue, it is skipped.

It cancels the active run and then starts, because a newer scheduled run takes priority over an older run that belongs to the same job: A new scheduled run does not preempt or cancel the existing active run to free its slot. The concurrency policy governs the new run's disposition.

References:

1. Automate jobs with schedules and triggers | Databricks on AWS:

<https://docs.databricks.com/aws/en/jobs/triggers>

Question 5

An ABAC column mask policy on catalog corp masks columns tagged pii. A new phone column in corp.crm.contacts holds personal data but was never tagged, and its values appear unmasked.

What fixes this with the least change?

- A. Attach the mask function with ALTER COLUMN SET MASK, since ABAC policies skip any column that was added to a table after the policy's creation
- B. Tag the phone column with pii, since the policy evaluates tags at query time and so applies to the column once the new tag takes effect on it
- C. Add the column's name to the policy's MATCH COLUMNS clause, since a policy lists each column that it masks explicitly by that column's name
- D. Recreate the policy after tagging the column, since a policy applies only to the columns that held the tag at the time the policy was created

Correct answer: B

B. Tag the phone column with pii, since the policy evaluates tags at query time and so applies to the column once the new tag takes effect on it

Explanation

Correct answer

Tag the phone column with pii, since the policy evaluates tags at query time and so applies to the column once the new tag takes effect on it

Why this is correct

An ABAC policy with a tag condition is evaluated against the governed tags present on the securable when access is evaluated. The new column is unmasked because it lacks the pii tag that matches the existing policy. Applying that tag makes the current policy cover the column without changing the policy definition, although a tag change can take a few minutes to take effect.

Why the other options are wrong

Attach the mask function with ALTER COLUMN SET MASK, since ABAC policies skip any column that was added to a table after the policy's creation - Attaching a separate mask manually would duplicate policy logic and is unnecessary when the ABAC condition already expresses the intended classification. The missing input is the tag.

Recreate the policy after tagging the column, since a policy applies only to the columns that held the tag at the time the policy was created - Tag-based evaluation reads governed metadata at access time, so a column tagged later can match the existing policy. Recreating the policy adds no needed behavior.

Add the column's name to the policy's MATCH COLUMNS clause, since a policy lists each column that it masks explicitly by that column's name - Policy matching is metadata-driven: the existing condition selects columns by tag. Assigning the governed pii tag makes the new column satisfy that condition.

References:

- [Create and manage row filter and column mask policies | Databricks on Google Cloud](#)

Question 6

A team needs an independent copy of a managed table in a sandbox schema for a month of experiments. VACUUM or changes on the source must never affect the copy, and the copy must not change the source.

Which statement fits?

- A. `CREATE TABLE sandbox.orders DEEP CLONE prod.orders`, which copies the data files and the metadata into a new, independent table of its own
- B. `CREATE TABLE sandbox.orders SHALLOW CLONE prod.orders`, which copies the metadata into a new table that keeps reading the source's data files
- C. `CREATE TABLE sandbox.orders LIKE prod.orders`, which copies the table's definition along with the data files that the table currently holds
- D. `CREATE VIEW sandbox.orders AS SELECT * FROM prod.orders`, which exposes the source table's current data under a new name in the sandbox

Correct answer: A

A. `CREATE TABLE sandbox.orders DEEP CLONE prod.orders`, which copies the data files and the metadata into a new, independent table of its own

Explanation

Correct answer:

`CREATE TABLE sandbox.orders DEEP CLONE prod.orders`, which copies the data files and the metadata into a new, independent table of its own

Why this is correct:

A deep clone copies the table's data files and metadata into a new table, so the sandbox copy has its own storage and does not depend on source files. Later source changes or VACUUM operations therefore do not change or invalidate the clone. This independence costs more storage than a shallow clone, which references the source files.

Why the other options are wrong:

`CREATE VIEW sandbox.orders AS SELECT * FROM prod.orders`, which exposes the source table's current data under a new name in the sandbox: A view stores a query definition and reads the source table's current data. It is not an independent copy, so source changes remain visible through it.

`CREATE TABLE sandbox.orders LIKE prod.orders`, which copies the table's definition along with the data files that the table currently holds: LIKE creates a table from the source definition but does not copy the existing table's data files. It cannot provide the month-long data snapshot requested.

`CREATE TABLE sandbox.orders SHALLOW CLONE prod.orders`, which copies the metadata into

a new table that keeps reading the source's data files: Because this clone references source data files instead of copying them, a source VACUUM can remove those files and make the clone unreadable.

References:

1. Clone a table on Databricks | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/delta/clone>

Question 7

A CI runner deploys a bundle to prod as a service principal. Security forbids personal access tokens, and the service principal's OAuth secret is kept in the CI system's secret store.

How should the runner authenticate the CLI?

- A. Set DATABRICKS_HOST, DATABRICKS_CLIENT_ID, and DATABRICKS_CLIENT_SECRET, so the CLI signs in with OAuth machine-to-machine authentication each run
- B. Set DATABRICKS_HOST and DATABRICKS_TOKEN to the OAuth secret, so the CLI exchanges the secret for a short-lived access token each time that it runs
- C. Run databricks auth login in the CI step, so the CLI opens a browser window and stores an OAuth token for the service principal to reuse later
- D. Set DATABRICKS_HOST and DATABRICKS_USERNAME to the service principal's application ID, so the CLI signs in with that identity for each deploy

Correct answer: A

A. Set DATABRICKS_HOST, DATABRICKS_CLIENT_ID, and DATABRICKS_CLIENT_SECRET, so the CLI signs in with OAuth machine-to-machine authentication each run

Explanation

Correct answer:

Set DATABRICKS_HOST, DATABRICKS_CLIENT_ID, and DATABRICKS_CLIENT_SECRET, so the CLI signs in with OAuth machine-to-machine authentication each run

Why this is correct:

Databricks CLI supports OAuth machine-to-machine authentication for unattended service-principal workflows. Supplying the workspace host, the service principal's client ID, and its OAuth client secret lets the CLI obtain short-lived access tokens without a personal access token or an interactive browser login. The secret should be injected from the CI secret store.

Why the other options are wrong:

Run databricks auth login in the CI step, so the CLI opens a browser window and stores an OAuth token for the service principal to reuse later: auth login is an interactive user-to-machine browser flow, which a headless CI runner cannot use as a service-principal login. M2M uses client credentials without interactive sign-in.

Set DATABRICKS_HOST and DATABRICKS_TOKEN to the OAuth secret, so the CLI exchanges the secret for a short-lived access token each time that it runs: An OAuth client secret is not a Databricks personal access token and cannot be passed as DATABRICKS_TOKEN. Configure it as the service principal client secret so the CLI performs OAuth.

Set DATABRICKS_HOST and DATABRICKS_USERNAME to the service principal's application ID, so the CLI signs in with that identity for each deploy: The application ID identifies the service principal but is not a username/password login credential. OAuth M2M requires both the client ID and its secret.

References:

1. Authentication for the Databricks CLI | Databricks on AWS:

<https://docs.databricks.com/aws/en/dev-tools/cli/authentication>

2. Environment variables and fields for unified authentication | Databricks on AWS:

<https://docs.databricks.com/aws/en/dev-tools/auth/env-vars>

Question 8

A silver table's null rate in `customer_email` has crept upward over several weeks without failing any pipeline rule. The team wants ongoing tracking of null rates and distribution drift for the table, with alerts.

Which feature fits?

- A. Predictive optimization on the table, which runs ANALYZE each night and alerts the table owner when a column's statistics drift over time
- B. A CHECK constraint on `customer_email`, which records the null rate of each write to the table and alerts once it drifts past a baseline
- C. Data profiling on the table, which computes metrics such as null fractions over time and compares them with a baseline to flag any drift
- D. An `expect_or_fail` rule on `customer_email`, which tracks the null rate across updates and raises an alert once it drifts past a baseline

Correct answer: C

C. Data profiling on the table, which computes metrics such as null fractions over time and compares them with a baseline to flag any drift

Explanation

Correct answer:

Data profiling on the table, which computes metrics such as null fractions over time and compares them with a baseline to flag any drift

Why this is correct:

Data profiling computes table metrics over time, including missing-value fractions and column distributions. Drift metrics compare a window with prior windows or a configured baseline, and profile metrics can be queried to create alerts when null rates exceed a threshold. This supports monitoring gradual quality changes without making every increase a pipeline failure.

Why the other options are wrong:

A CHECK constraint on `customer_email`, which records the null rate of each write to the table and alerts once it drifts past a baseline: A CHECK constraint accepts or rejects individual writes based on a row-level Boolean condition; it does not calculate historical null-rate trends or compare them to a baseline.

An `expect_or_fail` rule on `customer_email`, which tracks the null rate across updates and raises an alert once it drifts past a baseline: An expectation evaluates records during pipeline updates and can retain, drop, or fail on violating rows. It does not provide statistical drift tracking across time windows by itself.

Predictive optimization on the table, which runs ANALYZE each night and alerts the table owner when a column's statistics drift over time: Predictive optimization performs table maintenance such as OPTIMIZE, VACUUM, and ANALYZE. It is not a data-quality monitoring feature and does not alert on null-rate drift.

References:

1. Data profiling | Databricks on AWS:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/data-quality-monitoring/data-profiling>

2. Profile alerts | Databricks on AWS: <https://docs.databricks.com/aws/en/data-governance/unity-catalog/data-quality-monitoring/data-profiling/monitor-alerts>

Question 9

An engineer needs the customers that have at least one order, with only the customers' columns and exactly one row per customer, even when a customer has many rows in the orders DataFrame.

Which expression fits?

- A. `customers.join(orders, "customer_id", "inner")`
- B. `customers.join(orders, "customer_id", "left")`
- C. `customers.join(orders, "customer_id", "left_semi")`
- D. `customers.join(orders, "customer_id", "left_anti")`

Correct answer: C

C. `customers.join(orders, "customer_id", "left_semi")`

Explanation

Correct answer:

`customers.join(orders, "customer_id", "left_semi")`

Why this is correct:

A left-semi join returns rows and columns from the left DataFrame only when at least one matching right-side row exists. It behaves like an existence test, so multiple matching orders do not multiply a customer row. This satisfies both the customer-only schema and one-row-per-input-customer requirement, assuming customers itself is unique by customer ID.

Why the other options are wrong:

`customers.join(orders, "customer_id", "left")`: This join preserves customers without an order and repeats customers that match multiple orders, so it cannot implement the required existence test.

`customers.join(orders, "customer_id", "inner")`: An inner join excludes nonmatching customers but emits one result per matching pair. Customers with many orders therefore appear multiple times and order columns remain in the output.

`customers.join(orders, "customer_id", "left_anti")`: Only customers without an order are returned, which is the complement of the requested set.

References:

1. `pyspark.sql.DataFrame.join` | Apache Spark: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.join.html>

Question 10

A notebook's JDBC read from an Oracle table uses sound partitioning and a suitable number of connections, but each partition pulls rows in many tiny network round trips. The driver supports fetching larger batches.

Which reader option should be tuned?

- A. fetchsize, which sets how many rows the JDBC driver fetches per round trip, since a small driver default makes each partition read slowly
- B. numPartitions, which sets how many rows each connection fetches per round trip, since more partitions mean that each fetch returns a larger batch
- C. lowerBound, which sets how many rows the JDBC driver fetches per round trip, since a higher bound makes each partition's key range larger
- D. batchsize, which sets how many rows the JDBC driver fetches per round trip, since that one option controls both the reads and the writes

Correct answer: A

A. fetchsize, which sets how many rows the JDBC driver fetches per round trip, since a small driver default makes each partition read slowly

Explanation

Correct answer:

fetchsize, which sets how many rows the JDBC driver fetches per round trip, since a small driver default makes each partition read slowly

Why this is correct:

JDBC fetchsize controls the number of rows retrieved in each network round trip. A small fetch size forces many trips even when partitioning and connection parallelism are already appropriate; increasing it lets each partition fetch larger batches. It tunes batch transfer size rather than changing how work is divided across Spark partitions.

Why the other options are wrong:

batchsize, which sets how many rows the JDBC driver fetches per round trip, since that one option controls both the reads and the writes: Read-side transfer batching is controlled separately from write batching. Adjust the JDBC reader's fetchsize to reduce round trips.

numPartitions, which sets how many rows each connection fetches per round trip, since more partitions mean that each fetch returns a larger batch: numPartitions controls parallel JDBC connections/tasks, not the number of rows fetched per network request. The stem says the number of connections is already suitable.

lowerBound, which sets how many rows the JDBC driver fetches per round trip, since a higher bound makes each partition's key range larger: lowerBound helps define the partitioning range for a partition column; it does not set fetch batch size. Altering it could change partition ranges without fixing round trips.

References:

1. Lakehouse Federation performance recommendations | Databricks on AWS:

<https://docs.databricks.com/aws/en/query-federation/performance-recommendations>

Question 11

A join spends most of its time on one very frequent `customer_id`. Neither input fits in memory for a broadcast, AQE skew handling is not enough, and every matching pair of rows must be kept.

Which TWO approaches spread or isolate the hot key?

- A. Broadcast a list of the hot key values alone, so every executor knows which of its rows to skip while it processes its join partitions
- B. Process the hot key's rows in a separate join designed for their volume, then union that result with the join of all of the other keys
- C. Add more workers while keeping the join itself unchanged, so the single partition that holds the hot key is processed with more parallelism
- D. Salt the hot key with a random suffix on one side and replicate the matching rows on the other side, so the key's rows spread across partitions
- E. Remove the repeated `customer_id` values from both inputs before the join, so the hot key produces far fewer matching pairs in its partition

Correct answers: B, D

- B. Process the hot key's rows in a separate join designed for their volume, then union that result with the join of all of the other keys
- D. Salt the hot key with a random suffix on one side and replicate the matching rows on the other side, so the key's rows spread across partitions

Explanation

Correct answer:

Salt the hot key with a random suffix on one side and replicate the matching rows on the other side, so the key's rows spread across partitions

Process the hot key's rows in a separate join designed for their volume, then union that result with the join of all of the other keys

Why this is correct:

Salting adds multiple synthetic keys for a hot value on one side and replicates its matching rows on the other, distributing the hot join work across partitions while preserving every matching pair. A separate hot-key join is another valid strategy: handle the skewed key with a specialized path, then union its results with the ordinary join for other keys. Both avoid discarding matches.

Why the other options are wrong:

Add more workers while keeping the join itself unchanged, so the single partition that holds the hot key is processed with more parallelism: Adding workers does not split a single skewed partition

by itself; one task holding the hot key can remain the bottleneck. The join plan must redistribute or isolate that key.

Broadcast a list of the hot key values alone, so every executor knows which of its rows to skip while it processes its join partitions: Broadcasting only the key list does not make the large matching inputs broadcastable or distribute the hot-key join. It may identify rows but does not preserve all pairs through a balanced join.

Remove the repeated customer_id values from both inputs before the join, so the hot key produces far fewer matching pairs in its partition: Removing repeated keys discards matching rows and changes the join result. The question requires keeping every matching pair, so skew mitigation must preserve duplicates.

References:

1. Skew and spill | Databricks on AWS:

<https://docs.databricks.com/aws/en/optimizations/spark-ui-guide/long-spark-stage-page>

Question 12

A job defines the parameter `batch_date`. A notebook task expects an input named `load_date`, which must receive `batch_date`'s value on every run, including runs started with a different `batch_date`.

Which value should the task's `load_date` parameter use?

- A. `{{job.parameters.batch_date}}`, which resolves to the `batch_date` value of the current run, whether that value is the default or an override
- B. `${var.batch_date}`, which resolves to the `batch_date` value of the current run when the task starts, whether that value is the default or an override
- C. `{{input.batch_date}}`, which resolves to the `batch_date` value of the current run when the task starts, whether it is the default or an override
- D. `{{tasks.batch_date.values.load_date}}`, which resolves to the `batch_date` value of the current run, whether it is the default or an override

Correct answer: A

A. `{{job.parameters.batch_date}}`, which resolves to the `batch_date` value of the current run, whether that value is the default or an override

Explanation

Correct answer:

`{{job.parameters.batch_date}}`, which resolves to the `batch_date` value of the current run, whether that value is the default or an override

Why this is correct:

`{{job.parameters.batch_date}}` is a dynamic value reference resolved from the current job run's parameter values, including a value overridden at run time. It can be assigned to the notebook's `load_date` task parameter so the task receives the run-specific date rather than a deployment-time or unrelated task value.

Why the other options are wrong:

`${var.batch_date}`, which resolves to the `batch_date` value of the current run when the task starts, whether that value is the default or an override: `${var.batch_date}` is a bundle variable resolved when the bundle is deployed, not a job parameter resolved anew for each run. A run override would not change that deployment-time value.

`{{tasks.batch_date.values.load_date}}`, which resolves to the `batch_date` value of the current run, whether it is the default or an override: Task values are outputs explicitly set by an upstream task, and the syntax requires a task name and value key. `batch_date` is a job parameter, not a task result.

{{input.batch_date}}, which resolves to the **batch_date** value of the current run when the task starts, whether it is the default or an override: **{{input...}}** is used for values iterated by a For each task, not a top-level job parameter. No loop input exists in this scenario.

References:

1. Dynamic value references | Databricks on AWS:

<https://docs.databricks.com/aws/en/jobs/dynamic-value-references>

Question 13

A replayed feed repeats event_id values with different ingested_at timestamps, while every business field for an event_id is identical. Keeping any one copy per event_id is acceptable, and all columns must stay.

Which expression fits?

- A. `events.distinct()`, which keeps one row per event_id with all of its columns, as the repeated copies share the same business fields
- B. `events.dropDuplicates(["event_id"])`, which keeps one row per event_id with all of its columns, whichever of the copies that turns out to be
- C. `events.groupBy("event_id").count()`, which keeps one row per event_id and returns how many copies of each event the replayed feed delivered to the table
- D. `events.dropDuplicates(["ingested_at"])`, which keeps one row per ingestion timestamp, so each replayed copy of an event is removed

Correct answer: B

B. `events.dropDuplicates(["event_id"])`, which keeps one row per event_id with all of its columns, whichever of the copies that turns out to be

Explanation

Correct answer:

`events.dropDuplicates(["event_id"])`, which keeps one row per event_id with all of its columns, whichever of the copies that turns out to be

Why this is correct:

`dropDuplicates(["event_id"])` removes rows that share the specified key while retaining the rest of one representative row, including every other column. Since the question permits any copy and all business fields are identical, there is no need to choose a timestamp winner. The result has one row per event ID.

Why the other options are wrong:

`events.groupBy("event_id").count()`, which keeps one row per event_id and returns how many copies of each event the replayed feed delivered to the table: Grouping and counting collapses each event to an aggregate count and discards its other business columns. It answers how many copies exist rather than keeping a representative event row.

`events.dropDuplicates(["ingested_at"])`, which keeps one row per ingestion timestamp, so each replayed copy of an event is removed: Deduplicating on ingestion timestamp treats separate replay times as unique, so repeated event IDs can remain. The stated duplicate key is event_id.

`events.distinct()`, which keeps one row per event_id with all of its columns, as the repeated copies

share the same business fields: `distinct()` compares the entire row, including `ingested_at`. Since that timestamp differs between copies, the rows are distinct even though their event IDs and business fields match.

References:

1. **DataFrame.dropDuplicates | Apache Spark:** <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.dropDuplicates.html>

Question 14

Some records in a JSON feed are malformed. The batch must load the valid records and keep each malformed record's raw text in a column for later review, instead of failing or silently dropping it.

```
df = (spark.read.schema(schema_with_raw)
      .option("mode", _____)
      .option("columnNameOfCorruptRecord", "_raw")
      .json(path))
```

Which mode completes the reader?

- A. DROPMALFORMED, which loads the valid records and places the raw text of each dropped record in the `_raw` column so that it can be reviewed later
- B. PERMISSIVE, which loads the valid records and places the raw text of each malformed record in the `_raw` column so that it can be reviewed later
- C. RESCUE, which loads the valid records and places the raw text of each malformed record in the `_raw` column so that it can be reviewed later
- D. FAILFAST, which loads the valid records and places the raw text of the first malformed record in the `_raw` column before the whole read then stops

Correct answer: B

B. PERMISSIVE, which loads the valid records and places the raw text of each malformed record in the `_raw` column so that it can be reviewed later

Explanation

Correct answer:

PERMISSIVE, which loads the valid records and places the raw text of each malformed record in the `_raw` column so that it can be reviewed later

Why this is correct:

JSON parser PERMISSIVE mode keeps parsing valid records and places malformed input text in the field named by `columnNameOfCorruptRecord`, provided the supplied schema includes that string field. This lets the batch continue while retaining bad records for inspection. DROPMALFORMED discards them, and FAILFAST stops on malformed input.

Why the other options are wrong:

DROPMALFORMED, which loads the valid records and places the raw text of each dropped record in the `_raw` column so that it can be reviewed later: DROPMALFORMED skips corrupt rows rather than preserving their raw text in `_raw`. It is suitable only when discarding those records is acceptable.

FAILFAST, which loads the valid records and places the raw text of the first malformed record in the `_raw` column before the whole read then stops: FAILFAST raises an error when it encounters malformed input instead of returning a completed batch with a corrupt-record field. It does not preserve a partial result in the described way.

RESCUE, which loads the valid records and places the raw text of each malformed record in the `_raw` column so that it can be reviewed later: RESCUE is not a JSON parser mode for malformed syntax. Rescued data addresses schema or type mismatches; PERMISSIVE captures corrupt-record text.

References:

1. Spark API options reference | Databricks on AWS:

<https://docs.databricks.com/aws/en/spark/api-options>

Question 15

A notebook filters a large Spark DataFrame and then fails with driver out-of-memory errors at `toPandas()`. The executors show no memory problems, and the analysis only needs one summary row per region.

Which change fixes the immediate cause?

- A. Raise `spark.sql.shuffle.partitions`, so that the full result reaches the driver in more and smaller pieces that each fit in the driver's memory
- B. Aggregate by region in Spark first, and then call `toPandas` on the small result, so that only the summary rows ever reach the driver's memory
- C. Enable Arrow for the conversion to pandas, so the full result transfers to the driver in compact columnar batches that fit in its memory
- D. Increase the executors' memory, so that the conversion to pandas has more room on the workers while the full result is being collected

Correct answer: B

B. Aggregate by region in Spark first, and then call `toPandas` on the small result, so that only the summary rows ever reach the driver's memory

Explanation

Correct answer:

Aggregate by region in Spark first, and then call `toPandas` on the small result, so that only the summary rows ever reach the driver's memory

Why this is correct:

`toPandas()` collects the DataFrame's rows into the driver process, so a large filtered result can exhaust driver memory even when executors are healthy. Grouping and aggregating in Spark first reduces the distributed data to one row per region; only that small summary then crosses to the driver. This addresses the actual collection bottleneck rather than adding executor resources.

Why the other options are wrong:

Increase the executors' memory, so that the conversion to pandas has more room on the workers while the full result is being collected: The collection boundary is the driver process: it materializes the complete pandas result there. More worker memory cannot increase that process's available memory.

Raise `spark.sql.shuffle.partitions`, so that the full result reaches the driver in more and smaller pieces that each fit in the driver's memory: Shuffle partition count can change how distributed work is divided, but `toPandas()` still brings the entire final result to the driver. Smaller transfer pieces do not reduce its total memory requirement.

Enable Arrow for the conversion to pandas, so the full result transfers to the driver in compact columnar batches that fit in its memory: Arrow can improve conversion speed and transfer efficiency, but it does not make an arbitrarily large result fit in driver memory. Aggregate before collection to reduce the result size.

References:

1. Spark memory issues | Databricks on AWS:

<https://docs.databricks.com/aws/en/optimizations/spark-ui-guide/spark-memory-issues>

Question 16

Several bundle jobs must share one cluster definition, and each target may override it. The engineer declares a variable that holds the whole cluster mapping.

```
variables:
  job_cluster:
    description: Shared cluster spec
    _____
    default:
      spark_version: 15.4.x-scala2.12
      node_type_id: i3.xlarge
      num_workers: 2
```

Which line completes the declaration?

- A. type: complex, which allows the variable's default value to hold a whole mapping that the jobs then reference with `${var.job_cluster}`
- B. type: object, which allows the variable's default value to hold a whole mapping that the jobs then reference with `${var.job_cluster}`
- C. type: mapping, which allows the variable's default value to hold a whole mapping that the jobs then reference with `${var.job_cluster}`
- D. lookup: cluster, which allows the variable's default value to hold a whole mapping that the jobs then reference with `${var.job_cluster}`

Correct answer: A

A. type: complex, which allows the variable's default value to hold a whole mapping that the jobs then reference with `${var.job_cluster}`

Explanation

Correct answer:

type: complex, which allows the variable's default value to hold a whole mapping that the jobs then reference with `${var.job_cluster}`

Why this is correct:

Bundle custom variables default to simple strings; declaring type: complex allows the value to be a mapping or other structured configuration. That lets multiple jobs reference the whole cluster specification using `${var.job_cluster}`, while target-specific overrides can replace the mapping. complex is the supported type keyword for this purpose.

Why the other options are wrong:

lookup: cluster, which allows the variable's default value to hold a whole mapping that the jobs then reference with `${var.job_cluster}`: lookup resolves an existing named Databricks resource to its ID; it does

not declare an inline cluster mapping. A resource lookup cannot substitute for this structured default.

type: object, which allows the variable's default value to hold a whole mapping that the jobs then reference with `${var.job_cluster}`: object is not the supported bundle variable type keyword. Bundle configuration uses complex for a structured mapping.

type: mapping, which allows the variable's default value to hold a whole mapping that the jobs then reference with `${var.job_cluster}`: mapping is not a valid variable type declaration in Declarative Automation Bundles. The supported type for a whole mapping is complex.

References:

1. Substitutions and variables in Declarative Automation Bundles | Databricks on AWS:

<https://docs.databricks.com/aws/en/dev-tools/bundles/variables>

Question 17

Nightly OPTIMIZE with Z-ordering runs on classic job compute. Metrics show workers at full CPU for most of the run, low memory use, and ample storage throughput. The team will test one worker-family change at the same worker count.

Which family should it test?

- A. Memory optimized workers, which offer more memory per core, so each rewrite task can buffer more of its data before it writes out a new file
- B. GPU accelerated workers, which move the Parquet decoding and encoding for each rewritten file off the CPUs and onto the graphics processors instead
- C. Storage optimized workers, which offer faster local disks, so each rewrite task can read its source files from local storage instead of the cloud
- D. Compute optimized workers, which offer more CPU capacity per worker for the CPU-bound work of decoding, sorting, and rewriting the files

Correct answer: D

D. Compute optimized workers, which offer more CPU capacity per worker for the CPU-bound work of decoding, sorting, and rewriting the files

Explanation

Correct answer:

Compute optimized workers, which offer more CPU capacity per worker for the CPU-bound work of decoding, sorting, and rewriting the files

Why this is correct:

The measured bottleneck is sustained full CPU utilization, while memory and storage throughput have headroom. Z-ordering decodes, sorts, and rewrites files, so testing a compute-optimized worker family targets the resource that is saturated at the same worker count. The other families primarily add memory, GPU acceleration, or local storage rather than general CPU capacity.

Why the other options are wrong:

Memory optimized workers, which offer more memory per core, so each rewrite task can buffer more of its data before it writes out a new file: The observed workload is not memory-bound: utilization is low while CPU is saturated. Extra memory per core therefore does not target the measured bottleneck.

GPU accelerated workers, which move the Parquet decoding and encoding for each rewritten file off the CPUs and onto the graphics processors instead: GPU workers benefit workloads written to use GPU acceleration, such as supported machine-learning tasks. Ordinary Delta file decoding, sorting, and rewrite work is not automatically moved to GPUs.

Storage optimized workers, which offer faster local disks, so each rewrite task can read its source files from local storage instead of the cloud: Storage throughput is already ample, so adding local disk speed does not target the measured bottleneck. Z-ordering reads and writes cloud-backed Delta data.

References:

1. Compute configuration reference | Databricks on AWS:

<https://docs.databricks.com/aws/en/compute/configure>

Question 18

A report needs, for each depot, the total number of parcels and the number of parcels delivered late, computed in one pass over the parcels table.

```
SELECT depot, COUNT(*) AS parcels, _____ AS late_parcels
FROM parcels
GROUP BY depot
```

Which expression completes the query?

- A. `max(delivered_at > promised_at)`, which counts the rows in each depot where the delivery happened after the time that was promised
- B. `count(delivered_at > promised_at)`, which counts the rows in each depot where the delivery happened after the time that was promised
- C. `count_if(delivered_at > promised_at)`, which counts the rows in each depot where the delivery happened after the time that was promised
- D. `sum(delivered_at > promised_at)`, which counts the rows in each depot where the delivery happened after the time that was promised

Correct answer: C

C. `count_if(delivered_at > promised_at)`, which counts the rows in each depot where the delivery happened after the time that was promised

Explanation

Correct answer:

`count_if(delivered_at > promised_at)`, which counts the rows in each depot where the delivery happened after the time that was promised

Why this is correct:

`count_if` counts rows for which its boolean expression is TRUE within each group. The comparison `delivered_at > promised_at` marks late deliveries, so applying the aggregate alongside `COUNT(*)` produces both totals in one grouped scan. NULL comparisons do not count as TRUE.

Why the other options are wrong:

`max(delivered_at > promised_at)`, which counts the rows in each depot where the delivery happened after the time that was promised: `max` returns a boolean indicating whether any row is late, not the number of late rows. A depot with one late parcel and a depot with many would both yield TRUE.

`count(delivered_at > promised_at)`, which counts the rows in each depot where the delivery happened after the time that was promised: `COUNT(boolean_expression)` counts non-NULL results,

including both TRUE and FALSE. It therefore counts rows with known comparisons, not just late deliveries.

sum(delivered_at > promised_at), which counts the rows in each depot where the delivery happened after the time that was promised: Summing a boolean predicate is not the supported way to count matching records and can encounter boolean/numeric type issues. count_if directly expresses a count of TRUE values.

References:

1. count_if aggregate function - Azure Databricks - Databricks SQL:

https://learn.microsoft.com/en-us/azure/databricks/sql/language-manual/functions/count_if

Question 19

A job scheduled every hour rebuilds a report from corp.gold.sales, which changes only a few times a week at unpredictable hours. Most runs find no new data, and the report must refresh soon after each change.

Which trigger fits better?

- A. A table update trigger on corp.gold.sales, which starts the job after the table commits a change and so avoids the hourly empty executions
- B. A file arrival trigger on the table's storage location, which starts the job whenever the table writes a new data file to that location
- C. A schedule every 15 minutes, which starts the job four times as often, so the report refreshes sooner after each change to the table
- D. A continuous trigger, which keeps the job running all the time, so the report refreshes as soon as the table commits each of its changes

Correct answer: A

A. A table update trigger on corp.gold.sales, which starts the job after the table commits a change and so avoids the hourly empty executions

Explanation

Correct answer:

A table update trigger on corp.gold.sales, which starts the job after the table commits a change and so avoids the hourly empty executions

Why this is correct:

A table update trigger starts a job when the monitored Unity Catalog table receives a change commit. Since updates are infrequent and unpredictable, this avoids hourly runs that find no new data while still responding after each change. A scheduled trigger is appropriate when freshness must be checked on a fixed cadence instead.

Why the other options are wrong:

A schedule every 15 minutes, which starts the job four times as often, so the report refreshes sooner after each change to the table: A 15-minute schedule still runs repeatedly when the source is unchanged and can only bound freshness by polling frequency. Event-based table monitoring directly matches the source's irregular update times.

A continuous trigger, which keeps the job running all the time, so the report refreshes as soon as the table commits each of its changes: A continuous job reruns after each run completes or fails; it does not wait specifically for a table commit. It would run continuously even when the table rarely changes.

A file arrival trigger on the table's storage location, which starts the job whenever the table writes a new data file to that location: A file watcher observes object arrivals at a storage location, not commits in a table's transaction log. The source update here is a table event.

References:

1. Trigger jobs when source tables are updated | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/jobs/trigger-table-update>

Question 20

A notebook task named `validate` counts bad rows, and a later notebook task must read that count in the same job run. The engineer completes the two calls below.

```
# in validate
dbutils.jobs.taskValues._____
# in the later task
bad = dbutils.jobs.taskValues.get(taskKey="validate", key="bad_rows", default=0)
```

Which call completes the `validate` task?

- A. `put(key="bad_rows", value=bad_count)`, which stores the value under that key for the downstream tasks in the same job run to read later on
- B. `set(key="bad_rows", value=bad_count)`, which stores the value under that key for the downstream tasks in the same job run to read later on
- C. `set(key="bad_rows", value=bad_count, scope="job")`, which stores the value so that every later run of the job can read it back by that key
- D. `set(taskKey="validate", key="bad_rows", value=bad_count)`, which stores the value under that task and key for downstream tasks in the run to read

Correct answer: B

B. `set(key="bad_rows", value=bad_count)`, which stores the value under that key for the downstream tasks in the same job run to read later on

Explanation

Correct answer:

`set(key="bad_rows", value=bad_count)`, which stores the value under that key for the downstream tasks in the same job run to read later on

Why this is correct:

`dbutils.jobs.taskValues.set(key=..., value=...)` stores a named value produced by the current task for downstream tasks in the same job run. The later task can retrieve it by specifying the producer's task key and the same value key. The setter does not accept a taskKey or a persistent job scope because the current task is implied.

Why the other options are wrong:

`put(key="bad_rows", value=bad_count)`, which stores the value under that key for the downstream tasks in the same job run to read later on: The API method is named `set`, not `put`; the method name is part of the utility interface. Use the documented setter to publish a value.

`set(key="bad_rows", value=bad_count, scope="job")`, which stores the value so that every later

run of the job can read it back by that key: set does not take a scope argument or persist values for future job runs. Task values are transient communication for tasks within the current run.

set(taskKey="validate", key="bad_rows", value=bad_count), which stores the value under that task and key for downstream tasks in the run to read: The setter infers its producer from the executing task and accepts only key and value. taskKey is supplied by the downstream getter to identify which task produced the value.

References:

1. Databricks Utilities (dbutils) reference | Databricks on AWS:

<https://docs.databricks.com/aws/en/dev-tools/databricks-utils>

Question 21

A service principal must upload files into the volume `corp.landing.vendor_files` and read them back, but must not read any tables. It already holds `USE CATALOG` on `corp` and `USE SCHEMA` on `corp.landing`.

Which grants fit?

- A. `READ FILES` and `WRITE FILES` on the schema `corp.landing`, which let it read, add, and remove the files in that one volume and nothing else there
- B. `SELECT` and `MODIFY` on `corp.landing.vendor_files`, which let it read, add, and remove the files in that one volume and nothing else in the catalog
- C. `READ VOLUME` and `WRITE VOLUME` on `corp.landing.vendor_files`, which let it read, add, and remove files in that one volume and nothing else
- D. `WRITE VOLUME` alone on `corp.landing.vendor_files`, which lets it read, add, and remove the files in that one volume and nothing else in the catalog

Correct answer: C

C. `READ VOLUME` and `WRITE VOLUME` on `corp.landing.vendor_files`, which let it read, add, and remove files in that one volume and nothing else

Explanation

Correct answer:

`READ VOLUME` and `WRITE VOLUME` on `corp.landing.vendor_files`, which let it read, add, and remove files in that one volume and nothing else

Why this is correct:

Unity Catalog volume permissions use `READ VOLUME` and `WRITE VOLUME` to govern file operations on a volume. The service principal already has the required catalog and schema usage privileges, so those two object-level grants allow reading and writing files in this volume without granting table `SELECT` or `MODIFY` privileges elsewhere.

Why the other options are wrong:

`WRITE VOLUME` alone on `corp.landing.vendor_files`, which lets it read, add, and remove the files in that one volume and nothing else in the catalog: `WRITE VOLUME` permits creating, modifying, and deleting files but does not grant reading them. The task explicitly requires the principal to read files back as well.

`READ FILES` and `WRITE FILES` on the schema `corp.landing`, which let it read, add, and remove the files in that one volume and nothing else there: Those file privileges apply to external locations rather than a named volume. Volume access uses volume-scoped privileges.

SELECT and MODIFY on corp.landing.vendor_files, which let it read, add, and remove the files in that one volume and nothing else in the catalog: A volume is a file object, not a table that accepts SELECT and MODIFY privileges. Those grants do not express the required file access.

References:

1. Unity Catalog privileges reference | Databricks on AWS:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/access-control/privileges-reference>

Question 22

A column mask on corp.hr.staff.salary is no longer needed. The owner wants salary returned unmasked, the mask function kept for other tables, and all grants left unchanged.

Which statement fits?

- A. ALTER TABLE corp.hr.staff ALTER COLUMN salary DROP MASK, which detaches the mask from the column and leaves the UDF in place for reuse elsewhere
- B. REVOKE EXECUTE on the mask function from all users, which stops the mask from applying and returns the original values to every reader
- C. ALTER TABLE corp.hr.staff DROP ROW FILTER, which detaches every mask and filter on the table and returns the original values to readers
- D. DROP FUNCTION on the mask function, which detaches the mask from the salary column and returns the original values to every reader

Correct answer: A

A. ALTER TABLE corp.hr.staff ALTER COLUMN salary DROP MASK, which detaches the mask from the column and leaves the UDF in place for reuse elsewhere

Explanation

Correct answer:

ALTER TABLE corp.hr.staff ALTER COLUMN salary DROP MASK, which detaches the mask from the column and leaves the UDF in place for reuse elsewhere

Why this is correct:

ALTER COLUMN ... DROP MASK detaches a column mask from that table column while leaving the mask function itself defined. Other tables that use the same function keep their bindings, and no table grants are changed. This is the narrowest object operation for returning the salary value unmasked.

Why the other options are wrong:

ALTER TABLE corp.hr.staff DROP ROW FILTER, which detaches every mask and filter on the table and returns the original values to readers: Dropping a row filter removes row-level filtering, not a mask bound to one column. It could also alter visibility for every row without detaching this column policy.

DROP FUNCTION on the mask function, which detaches the mask from the salary column and returns the original values to every reader: Dropping the function would remove a reusable object and can fail while other columns depend on it. Detach the specific binding instead.

REVOKE EXECUTE on the mask function from all users, which stops the mask from applying and returns the original values to every reader: Revoking function execution does not remove the column's mask binding; it can cause authorization or evaluation problems rather than restore ordinary

reads. Grants are also meant to remain unchanged.

References:

1. Manually apply row filters and column masks | Databricks on AWS:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/filters-and-masks/manually-apply>

Question 23

Partners should query only three columns of a governed orders table through a SQL warehouse. They must not be granted SELECT on the table itself, and results must always reflect the current data.

Which gold object fits?

A. A materialized view that selects the three columns, with SELECT granted on it, since it reflects the table's current data at every moment

B. A view that selects the three columns, with SELECT granted on the view, since partners then also receive SELECT on the base table itself

C. A temporary view that selects the three columns, created in each partner's session, since it grants read access for that session only

D. A view that selects the three columns, with SELECT granted on the view, since the view owner's access lets the partners read through it

Correct answer: D

D. A view that selects the three columns, with SELECT granted on the view, since the view owner's access lets the partners read through it

Explanation

Correct answer:

A view that selects the three columns, with SELECT granted on the view, since the view owner's access lets the partners read through it

Why this is correct:

A standard view stores a query definition and can expose a controlled projection to users granted SELECT on the view. With Unity Catalog view ownership and access delegation configured, partners can query the three selected columns without receiving SELECT on the underlying table directly. Because the view queries the current base data, its results reflect current values when queried.

Why the other options are wrong:

A temporary view that selects the three columns, created in each partner's session, since it grants read access for that session only: A temporary view exists only in a session and is not a durable shared interface for partners. It also does not grant persistent access through a controlled object.

A view that selects the three columns, with SELECT granted on the view, since partners then also receive SELECT on the base table itself: The purpose of the view is to expose a limited projection while keeping the source table's grants unchanged. Grant SELECT on the view to the partner group.

A materialized view that selects the three columns, with SELECT granted on it, since it reflects the

table's current data at every moment: A materialized view stores refreshed results and can lag between refreshes; it does not necessarily reflect changes at every moment. A regular view evaluates against the current base data at query time.

References:

1. Views | Databricks on AWS: <https://docs.databricks.com/aws/en/views>

Question 24

A notebook reads a large PostgreSQL orders table over JDBC. No numeric or date column suits partitioning, but the region column holds five known values, and the read should run as five parallel queries, one per region.

```
orders = spark.read.jdbc(  
    url, "orders",  
    _____,  
    properties=props)
```

Which argument completes the read?

- A. column="region", lowerBound="AMER", upperBound="EMEA", numPartitions=5, so Spark splits the region values into five ranges that it reads in parallel
- B. numPartitions=5 without a partition column, so Spark divides the orders table into five equal slices and then runs one parallel query per slice
- C. fetchsize=5, so Spark opens five JDBC connections, one for each region value, and each connection reads the rows of its own region in parallel
- D. predicates=["region = 'AMER'", "region = 'APAC'", ...] with one condition per region, so each condition becomes its own partition and query

Correct answer: D

D. predicates=["region = 'AMER'", "region = 'APAC'", ...] with one condition per region, so each condition becomes its own partition and query

Explanation

Correct answer

predicates=["region = 'AMER'", "region = 'APAC'", ...] with one condition per region, so each condition becomes its own partition and query

Why this is correct

The predicates argument of `spark.read.jdbc` takes a list of WHERE-clause conditions, and each condition defines one partition of the resulting DataFrame. Supplying one condition per region makes Spark send five separate queries in parallel, which works even though the table has no numeric, date, or timestamp column for range partitioning.

Why the other options are wrong:

column="region", lowerBound="AMER", upperBound="EMEA", numPartitions=5, so Spark splits the region values into five ranges that it reads in parallel - Range partitioning needs a numeric, date,

or timestamp partition column, and the bounds only set the stride of those ranges. A string region column cannot be split this way.

numPartitions=5 without a partition column, so Spark divides the orders table into five equal slices and then runs one parallel query per slice - Without a partition column and bounds, or a predicates list, Spark has nothing to split the query on, so the read runs as a single query. numPartitions alone does not slice a table.

fetchsize=5, so Spark opens five JDBC connections, one for each region value, and each connection reads the rows of its own region in parallel - fetchsize sets how many rows the driver returns per network round trip. It creates no partitions, and a value of 5 would make the read slower.

References:

- [pyspark.sql.DataFrameReader.jdbc | Apache Spark](#)

- [JDBC To Other Databases | Apache Spark](#)

Question 25

A sales DataFrame has one row per store, month, and amount. A report needs one row per store with one column per month for Jan, Feb, and Mar, each holding that month's total amount.

Which expression fits?

- A. `sales.groupBy("store").pivot("month", ["Jan", "Feb", "Mar"]).count()`
- B. `sales.groupBy("store").pivot("month", ["Jan", "Feb", "Mar"]).sum("amount")`
- C. `sales.groupBy("month").pivot("store").sum("amount")`
- D. `sales.groupBy("store", "month").sum("amount")`

Correct answer: B

B. `sales.groupBy("store").pivot("month", ["Jan", "Feb", "Mar"]).sum("amount")`

Explanation

Correct answer:

`sales.groupBy("store").pivot("month", ["Jan", "Feb", "Mar"]).sum("amount")`

Why this is correct:

`pivot("month", [...])` turns the selected month values into output columns after grouping by store. Aggregating `sum("amount")` combines any multiple store-month rows into the requested monthly total. Supplying the month list also fixes the output columns and their intended categories.

Why the other options are wrong:

`sales.groupBy("month").pivot("store").sum("amount")`: That layout puts stores across columns and months down the rows, the reverse of the report's requested dimensions.

`sales.groupBy("store", "month").sum("amount")`: Keeping both dimensions as grouping keys leaves month values in rows instead of turning them into columns. The report needs a wide pivoted layout.

`sales.groupBy("store").pivot("month", ["Jan", "Feb", "Mar"]).count()`: `count()` counts source rows for each store-month rather than summing their amounts. The grouping and pivot are appropriate, but the aggregation is not.

References:

1. **GroupedData.pivot | Apache Spark:** <https://spark.apache.org/docs/latest/api/python/reference/pyspa>

rk.sql/api/pyspark.sql.GroupedData.pivot.html

Question 26

A workspace allows only serverless compute. The team wants managed Lakeflow Connect ingestion from Salesforce and, through the standard gateway-based connector, from an on-premises SQL Server database.

Which statement is correct?

- A. The SQL Server connector runs fully on serverless compute, while the Salesforce connector needs a gateway on classic compute to reach its API
- B. Neither connector can run there, as every managed connector needs an ingestion gateway on classic compute to read from its source system
- C. The Salesforce connector runs fully on serverless compute, while the SQL Server connector needs an ingestion gateway that runs on classic compute
- D. Both connectors run fully on serverless compute, as each managed connector places its own ingestion gateway on serverless compute as well

Correct answer: C

C. The Salesforce connector runs fully on serverless compute, while the SQL Server connector needs an ingestion gateway that runs on classic compute

Explanation

Correct answer:

The Salesforce connector runs fully on serverless compute, while the SQL Server connector needs an ingestion gateway that runs on classic compute

Why this is correct:

Managed SaaS connectors such as Salesforce run on serverless infrastructure. Database CDC connectors use an ingestion gateway to connect to the source; that gateway requires classic compute, even though the ingestion pipeline runs on serverless. In a serverless-only workspace, Salesforce can run but the standard SQL Server gateway-based connector cannot be deployed.

Why the other options are wrong:

The SQL Server connector runs fully on serverless compute, while the Salesforce connector needs a gateway on classic compute to reach its API: The compute requirements are reversed: Salesforce is the SaaS serverless connector, while SQL Server requires a database ingestion gateway. The gateway is not needed to reach Salesforce's API.

Neither connector can run there, as every managed connector needs an ingestion gateway on classic compute to read from its source system: SaaS-source connectors run fully on serverless, without an ingestion gateway. Gateway requirements apply to database CDC connectors such as SQL Server.

Both connectors run fully on serverless compute, as each managed connector places its own ingestion gateway on serverless compute as well: The SQL Server gateway cannot run on serverless compute in this configuration. A serverless ingestion pipeline does not make the separate gateway serverless.

References:

1. Managed connector FAQs | Databricks on AWS:

<https://docs.databricks.com/aws/en/ingestion/lakeflow-connect/faq>

Question 27

An ingestion notebook reads scanned PDF contracts from a volume with the `binaryFile` source. A later process will extract the text, so this step only needs to capture each document and basic facts about it.

What does the reader produce?

- A.** One row per folder, holding each file's bytes collected into an array column, together with the folder's path and its total file count
- B.** One row per file, holding the file's path, modification time, and length, plus its raw bytes in a binary content column for later parsing
- C.** One row per page of each file, holding the page's text extracted into a string column, with the page's images kept in a separate binary column
- D.** One row per file, holding its text already extracted into a string column, with the file's path and length kept alongside it as metadata

Correct answer: B

B. One row per file, holding the file's path, modification time, and length, plus its raw bytes in a binary content column for later parsing

Explanation

Correct answer:

One row per file, holding the file's path, modification time, and length, plus its raw bytes in a binary content column for later parsing

Why this is correct:

Spark's `binaryFile` source produces one record per file, with path, `modificationTime`, length, and content fields. The content is the raw bytes; the source does not parse a PDF into pages or extract its text. A later parser can consume those bytes while this ingestion step preserves the original document and its file metadata.

Why the other options are wrong:

One row per page of each file, holding the page's text extracted into a string column, with the page's images kept in a separate binary column: Page splitting and OCR/text extraction require a document-processing step, not the binary file data source. `binaryFile` returns the whole file's bytes as one record.

One row per file, holding its text already extracted into a string column, with the file's path and length kept alongside it as metadata: The content field remains binary, not extracted text. Reading raw bytes is what allows a later process to select its own PDF parser.

One row per folder, holding each file's bytes collected into an array column, together with the folder's path and its total file count: The source emits one row for each file rather than grouping files into folders or collecting their bytes into an array. Folder-level aggregation would require separate transformations.

References:

1. Binary File Data Source | Apache Spark:

<https://spark.apache.org/docs/3.5.3/sql-data-sources-binaryFile.html>

Question 28

A silver Delta table receives inserts, updates, and deletes, and change data feed is enabled on it. A downstream stream must process each row-level change, including deletes, starting from table version 120.

```
changes = (spark.readStream
  .option(_____)
  .option("startingVersion", 120)
  .table("corp.silver.orders"))
```

Which option completes the reader?

- A. "cloudFiles.format", "delta", which returns each change file of the table as a row, with an extra column that describes the file's operation
- B. "readChangeFeed", "true", which returns each row-level change with a `_change_type` column such as insert, update_postimage, or delete for it
- C. "ignoreDeletes", "true", which returns the inserts and updates as rows and records each delete in a separate column of the stream's output
- D. "skipChangeCommits", "true", which returns each insert as a row and passes the updates and deletes through as rows that are marked with their type

Correct answer: B

B. "readChangeFeed", "true", which returns each row-level change with a `_change_type` column such as insert, update_postimage, or delete for it

Explanation

Correct answer:

"readChangeFeed", "true", which returns each row-level change with a `_change_type` column such as insert, update_postimage, or delete for it

Why this is correct:

Setting `readChangeFeed=true` makes the streaming reader consume row-level Delta change data rather than only the table's append snapshot. The configured `startingVersion` begins the feed at version 120, and output includes `_change_type` values such as insert, update preimage/postimage, and delete. The downstream stream can then handle every change explicitly.

Why the other options are wrong:

"skipChangeCommits", "true", which returns each insert as a row and passes the updates and deletes through as rows that are marked with their type: `skipChangeCommits` ignores update and delete commits and processes only appends. It does not emit row-level change events or their types.

"ignoreDeletes", "true", which returns the inserts and updates as rows and records each delete in a separate column of the stream's output: ignoreDeletes is a legacy option for partition-boundary deletes and does not provide update or delete records. It cannot meet the requirement to process every row-level change.

"cloudFiles.format", "delta", which returns each change file of the table as a row, with an extra column that describes the file's operation: Auto Loader format options describe file ingestion, not Delta table change events. The Delta reader needs readChangeFeed to expose row-level events.

References:

1. Use change data feed on Databricks | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/tables/features/change-data-feed>

Question 29

An engineer made uncommitted edits to one notebook in a Git folder and wants to throw them away. Other people have pushed commits to the same remote branch, and those commits must not be affected.

What should the engineer do?

- A. Use Reset in the Git folder dialog, which restores the local notebook to its last committed version and leaves the remote branch untouched
- B. Pull the remote branch, which replaces the uncommitted edits in the notebook with the latest committed version from the remote branch
- C. Discard the changes to that notebook in the Git folder dialog, which restores its last committed version and leaves the remote untouched
- D. Switch to another branch and back again, which drops the uncommitted edits to the notebook and restores its last committed version

Correct answer: C

C. Discard the changes to that notebook in the Git folder dialog, which restores its last committed version and leaves the remote untouched

Explanation

Correct answer:

Discard the changes to that notebook in the Git folder dialog, which restores its last committed version and leaves the remote untouched

Why this is correct:

The Git folder's Discard action reverts the selected notebook's working-tree edits to its last committed version. It changes only the local uncommitted file and does not push, reset, or modify the shared remote branch. That is the targeted way to throw away one notebook's local edits.

Why the other options are wrong:

Pull the remote branch, which replaces the uncommitted edits in the notebook with the latest committed version from the remote branch: Pulling integrates remote commits and may leave local modifications in place or create conflicts; it is not the command to discard selected local changes. It can also bring in teammates' commits without reverting the notebook.

Switch to another branch and back again, which drops the uncommitted edits to the notebook and restores its last committed version: Branch switching may be blocked by uncommitted changes and is not a reliable discard operation. The explicit discard action targets the edited notebook directly.

Use Reset in the Git folder dialog, which restores the local notebook to its last committed version

and leaves the remote branch untouched: A branch reset targets repository history/state rather than one selected notebook. The file's Discard command directly reverts its uncommitted working-tree content.

References:

1. Git folders | Databricks on AWS: <https://docs.databricks.com/aws/en/repos/git-operations-with-repos>

Question 30

Queries on an ABAC-protected table succeed on a SQL warehouse, but the same query from a dedicated cluster on Databricks Runtime 15.4 LTS fails with an access error. The user's privileges are identical on both.

What should the team do?

- A. Remove the table's governed tags while working on the cluster, since dedicated clusters read ABAC-secured tables only when they are untagged
- B. Grant the user EXECUTE on the policy's function, since dedicated clusters evaluate each policy with the querying user's own privileges
- C. Use serverless compute, or Runtime 16.4 or later with fine-grained access control on for dedicated compute, since earlier runtimes cannot read ABAC-secured tables
- D. Grant the user MANAGE on the table, since dedicated clusters read ABAC-secured tables only for users who are able to manage the policies

Correct answer: C

C. Use serverless compute, or Runtime 16.4 or later with fine-grained access control on for dedicated compute, since earlier runtimes cannot read ABAC-secured tables

Explanation

Correct answer

Use serverless compute, or Runtime 16.4 or later with fine-grained access control on for dedicated compute, since earlier runtimes cannot read ABAC-secured tables

Why this is correct

ABAC-secured tables require serverless compute, standard compute on Databricks Runtime 16.4 or later, or dedicated compute on 16.4 or later with fine-grained access control filtering enabled. Dedicated compute on Runtime 15.4 LTS predates that support, so identical user privileges cannot resolve the runtime access error. Moving the query to serverless compute, or upgrading the dedicated cluster to 16.4 or later with fine-grained access control enabled, addresses the compatibility issue.

Why the other options are wrong

Remove the table's governed tags while working on the cluster, since dedicated clusters read ABAC-secured tables only when they are untagged - Removing classification tags would bypass the intended policy rather than fix unsupported runtime enforcement. It would also expose sensitive columns.

Grant the user EXECUTE on the policy's function, since dedicated clusters evaluate each policy with the querying user's own privileges - The query succeeds on a SQL warehouse with the same user privileges, so the difference is not missing EXECUTE permission. The dedicated runtime is below

the required ABAC version.

Grant the user MANAGE on the table, since dedicated clusters read ABAC-secured tables only for users who are able to manage the policies - MANAGE is an administrative privilege and is not needed to query a protected table. The runtime support discrepancy applies regardless of table-management privileges.

References:

- [Row filter and column mask policy evaluation and runtime behavior | Databricks on Google Cloud](#)
- [Requirements for ABAC | Databricks on AWS](#)

Question 31

A continuous job processes events and takes several minutes per run. To cut latency, the team asks whether raising the job's maximum concurrent runs would let a second run start while the first is still working.

What is the answer?

- A. No, as a continuous job has a single running instance, and the next run of the job starts only after the previous run of the job has ended
- B. No, as overlapping runs need a file arrival trigger instead, which then starts additional runs of the job as the new events arrive
- C. Yes, but only with queueing enabled, as queued continuous runs start as soon as the job's concurrency limit allows another run of it
- D. Yes, as a higher concurrency limit lets the continuous trigger start another run of the job before the previous run of the job has ended

Correct answer: A

A. No, as a continuous job has a single running instance, and the next run of the job starts only after the previous run of the job has ended

Explanation

Correct answer:

No, as a continuous job has a single running instance, and the next run of the job starts only after the previous run of the job has ended

Why this is correct:

A continuous job is designed to maintain one active run and start another when the current run completes or fails. Raising `max_concurrent_runs` does not create overlapping continuous instances, so it cannot reduce latency by starting a second copy while the first is still processing. Faster processing or a different streaming architecture would be needed to lower event latency.

Why the other options are wrong:

No, as overlapping runs need a file arrival trigger instead, which then starts additional runs of the job as the new events arrive: Another trigger type does not make an existing continuous job overlap itself. A file-arrival trigger would not change the continuous job's single-running-instance behavior.

Yes, but only with queueing enabled, as queued continuous runs start as soon as the job's concurrency limit allows another run of it: Queueing controls pending runs for supported job triggers, but continuous execution still maintains its single active run. It does not launch a concurrent continuous instance when capacity becomes available.

Yes, as a higher concurrency limit lets the continuous trigger start another run of the job before the previous run of the job has ended: The maximum concurrency setting does not override continuous-trigger semantics. A continuous job starts its next run after the active one ends.

References:

1. Automate jobs with schedules and triggers | Databricks on AWS:

<https://docs.databricks.com/aws/en/jobs/triggers>

Question 32

Devices publish to Kafka topics named `device_` followed by a region code, and new regions add new topics over time. One stream must read all of these topics, including ones created after it starts, with no code changes.

Which reader option fits?

- A. `subscribe` set to a comma-separated list of the current topics, which the stream then extends on its own whenever a new matching topic appears
- B. `subscribePattern` set to `device_.*`, which subscribes to every topic that matches the pattern, including any topic created after the stream starts
- C. `assign` set to `device_.*`, which assigns the partitions of each matching topic to the stream and picks up newly created topics on the next restart
- D. `subscribe` set to `device_.*`, which treats its value as a regular expression and subscribes to each matching topic, including topics created later

Correct answer: B

B. `subscribePattern` set to `device_.*`, which subscribes to every topic that matches the pattern, including any topic created after the stream starts

Explanation

Correct answer:

`subscribePattern` set to `device_.*`, which subscribes to every topic that matches the pattern, including any topic created after the stream starts

Why this is correct:

Kafka's `subscribePattern` option accepts a regular expression and subscribes to topics matching that pattern. Unlike enumerating current topic names, a pattern subscription also discovers newly created topics that match while the streaming query runs. That allows regions to be added without editing the reader configuration.

Why the other options are wrong:

subscribe set to a comma-separated list of the current topics, which the stream then extends on its own whenever a new matching topic appears: An explicit `subscribe` list contains the named topics, not a rule for future topic names. New regions would require updating the list.

subscribe set to `device_.*`, which treats its value as a regular expression and subscribes to each matching topic, including topics created later: The `subscribe` option accepts topic names, not a regular expression. Use the dedicated `pattern` option to treat `device_.*` as a regex.

assign set to `device_.*`, which assigns the partitions of each matching topic to the stream and

picks up newly created topics on the next restart: assign specifies exact topic-partition assignments rather than a topic-name pattern. It does not discover matching future topics from a regex.

References:

1. Structured Streaming + Kafka Integration Guide (Kafka broker version 0.10.0 or higher) | Apache Spark: <https://spark.apache.org/docs/3.5.6/structured-streaming-kafka-integration.html>

Question 33

A bundle must build the project's Python wheel during deployment and install it on a job task. The task side is already set up to use whatever wheel the build produces.

```
resources:
  jobs:
    etl:
      tasks:
        - task_key: main
          libraries:
            - whl: ./dist/*.whl
```

Which top-level block completes the configuration?

- A. include: {default: {type: whl, build: poetry build, path: .}}, which builds the project's wheel into ./dist each time that the bundle deploys
- B. artifacts: {default: {type: whl, build: poetry build, path: .}}, which builds the project's wheel into ./dist each time that the bundle deploys
- C. variables: {default: {type: whl, build: poetry build, path: .}}, which builds the project's wheel into ./dist each time that the bundle deploys
- D. sync: {default: {type: whl, build: poetry build, path: .}}, which builds the project's wheel into ./dist each time that the bundle deploys

Correct answer: B

B. artifacts: {default: {type: whl, build: poetry build, path: .}}, which builds the project's wheel into ./dist each time that the bundle deploys

Explanation

Correct answer:

artifacts: {default: {type: whl, build: poetry build, path: .}}, which builds the project's wheel into ./dist each time that the bundle deploys

Why this is correct:

Declarative Automation Bundles use the top-level artifacts mapping to define buildable artifacts such as Python wheels. The whl artifact can specify a build command and project path; deploying the bundle builds the wheel into the configured distribution output so the job task can install the resulting file. Resource library configuration then points to that artifact.

Why the other options are wrong:

include: {default: {type: whl, build: poetry build, path: .}}, which builds the project's wheel into ./dist each time that the bundle deploys: include selects configuration files to load into the bundle; it does not define

a build artifact or run Poetry. The wheel build belongs in artifacts.

variables: {default: {type: whl, build: poetry build, path: .}}, which builds the project's wheel into ./dist each time that the bundle deploys: variables defines values for configuration substitution, not package build instructions. It cannot produce the wheel referenced by the task.

sync: {default: {type: whl, build: poetry build, path: .}}, which builds the project's wheel into ./dist each time that the bundle deploys: sync controls project file synchronization to the workspace, not package compilation. Artifact build configuration belongs in the dedicated block.

References:

1. Configuration reference | Databricks on AWS:

<https://docs.databricks.com/aws/en/dev-tools/bundles/reference>

Question 34

A job has three tasks: fetch, then reconcile, then record_status. reconcile fails, and record_status uses Run if All done and succeeds. No task is canceled or excluded in the run.

Which result does the run show?

- A. Succeeded, as the last task in the run succeeded, and a job run's overall result follows only the result of the final task that it runs
- B. Canceled, as a failed task cancels the whole run at that point, and record_status then runs as a cleanup step outside of that job run
- C. Failed, as a job run that contains any failed task is always marked as failed, whatever the results of its leaf tasks turn out to be in the end
- D. Succeeded with failures, as some of its tasks failed while every leaf task in the job run completed successfully in the end

Correct answer: D

D. Succeeded with failures, as some of its tasks failed while every leaf task in the job run completed successfully in the end

Explanation

Correct answer:

Succeeded with failures, as some of its tasks failed while every leaf task in the job run completed successfully in the end

Why this is correct:

Lakeflow Jobs determines the overall result from the job's task outcomes, not only the final task. When a task fails but the downstream leaf task is allowed to run and succeeds, the run can finish as Succeeded with failures, indicating successful completion of the workflow path with an earlier task failure retained. All done allows the status task to run after reconcile regardless of its result.

Why the other options are wrong:

Succeeded, as the last task in the run succeeded, and a job run's overall result follows only the result of the final task that it runs: The last task's success does not erase a failed task earlier in the run. Job result states retain the failure while recognizing that the eligible leaf work completed.

Canceled, as a failed task cancels the whole run at that point, and record_status then runs as a cleanup step outside of that job run: The overall result depends on leaf tasks, so the status task's run-if condition matters: All done lets it execute after reconcile fails.

Failed, as a job run that contains any failed task is always marked as failed, whatever the results of its leaf tasks turn out to be in the end: An earlier failed task does not always force the overall result

to Failed when the job's terminal tasks complete successfully under the run-if design. The platform distinguishes succeeded-with-failures from outright failure.

References:

1. Jobs system table reference | Databricks on AWS:

<https://docs.databricks.com/aws/en/admin/system-tables/jobs>

Question 35

A company runs two workspaces in us-east-1 and plans a third in eu-west-1. Analysts in the EU workspace must read a few curated tables governed by the US workspaces' Unity Catalog metastore.

How should this be set up?

- A. Attach the EU workspace to a metastore in eu-west-1, and have the US metastore share the curated tables with that metastore through Delta Sharing
- B. Attach the EU workspace to a new metastore in eu-west-1, then grant the EU analysts SELECT on the US tables directly across the two metastores
- C. Attach all three workspaces to a new metastore in eu-west-1, as a metastore must sit in the same region as the most recently added workspace
- D. Attach the EU workspace to the existing US metastore, as one metastore can govern workspaces in any region of the same Databricks account

Correct answer: A

A. Attach the EU workspace to a metastore in eu-west-1, and have the US metastore share the curated tables with that metastore through Delta Sharing

Explanation

Correct answer:

Attach the EU workspace to a metastore in eu-west-1, and have the US metastore share the curated tables with that metastore through Delta Sharing

Why this is correct:

Unity Catalog metastores are regional: a workspace attaches to the metastore in its region. Cross-region table access should use Databricks-to-Databricks OpenSharing (Delta Sharing), which lets the US provider share curated tables to the EU metastore without registering one managed table in two metastores. This preserves the regional governance boundary.

Why the other options are wrong:

Attach all three workspaces to a new metastore in eu-west-1, as a metastore must sit in the same region as the most recently added workspace: Regional workspaces can remain attached to their regional metastores; Delta Sharing provides the cross-region data exchange needed here.

Attach the EU workspace to the existing US metastore, as one metastore can govern workspaces in any region of the same Databricks account: Metastore assignment is regional, so an EU workspace must use the metastore in its own region.

Attach the EU workspace to a new metastore in eu-west-1, then grant the EU analysts SELECT on

the US tables directly across the two metastores: Metastore privileges do not cross regional boundaries as direct grants on another metastore's table. Share the data to the EU metastore and grant access on its recipient-side share/catalog.

References:

1. Unity Catalog best practices | Databricks on AWS:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/best-practices>

Question 36

Analysts run many queries on one classic cluster that scan the same few Delta tables repeatedly during the day. Each scan pulls the same remote files again, and the cluster's workers have local SSDs.

Which setting should be tested first, then remeasured?

- A. Call `cache()` on each query's result `DataFrame`, so that later scans of the same tables read those cached results from executor memory instead
- B. Raise `spark.sql.files.maxPartitionBytes`, so each scan reads the same remote files in fewer and larger partitions and finishes a little sooner
- C. Enable the disk cache, so the data files read from cloud storage are kept on the workers' local SSDs and later scans read those local copies
- D. Enable predictive optimization, so the files that each scan reads are compacted into fewer objects that are then fetched only once per day

Correct answer: C

C. Enable the disk cache, so the data files read from cloud storage are kept on the workers' local SSDs and later scans read those local copies

Explanation

Correct answer:

Enable the disk cache, so the data files read from cloud storage are kept on the workers' local SSDs and later scans read those local copies

Why this is correct:

Databricks disk cache stores copies of remote Parquet data files on worker-local storage and serves later reads from those local copies. With SSD-backed workers repeatedly scanning the same Delta files, enabling disk cache is a targeted first experiment. It differs from Spark `cache()`, which manually stores a `DataFrame` or `RDD` in memory.

Why the other options are wrong:

Raise `spark.sql.files.maxPartitionBytes`, so each scan reads the same remote files in fewer and larger partitions and finishes a little sooner: Larger file partitions may reduce task count, but they do not prevent repeated remote reads of the same data files. The stated bottleneck is repeated storage access.

Enable predictive optimization, so the files that each scan reads are compacted into fewer objects that are then fetched only once per day: Predictive optimization performs maintenance and statistics collection; it does not cache each source file locally for successive reads on the cluster. It is not a once-a-day fetch mechanism.

Call `cache()` on each query's result DataFrame, so that later scans of the same tables read those cached results from executor memory instead: Spark `cache()` stores a specific computed DataFrame in memory and requires code changes; it does not automatically cache table data files for unrelated scans. Disk cache targets repeated file reads.

References:

1. Optimize performance with caching on Databricks | Databricks on AWS:

<https://docs.databricks.com/aws/en/optimizations/disk-cache>

Question 37

A job cluster fails to start with a termination reason reporting insufficient instance capacity in its availability zone. The same cluster definition started normally the day before, and nothing in it has changed.

Which response fits?

- A. Retry with another instance type or availability zone, or allow a fallback, since the cloud provider lacked capacity for that instance type
- B. Remove the cluster's init scripts, since an insufficient capacity error means a script used up the instance's resources during startup
- C. Pin older versions of the job's libraries, since an insufficient capacity error means a library conflict stopped the workers from joining
- D. Request a higher vCPU quota from the cloud provider, since the error means the account has reached its limit on how many instances it can run

Correct answer: A

A. Retry with another instance type or availability zone, or allow a fallback, since the cloud provider lacked capacity for that instance type

Explanation

Correct answer:

Retry with another instance type or availability zone, or allow a fallback, since the cloud provider lacked capacity for that instance type

Why this is correct:

An insufficient-capacity termination indicates the cloud provider could not allocate the requested instance type in that availability zone at that time. This can happen even when the same configuration worked previously and quotas are available. Retrying with another type or zone, or configuring fallback capacity, gives the scheduler alternatives.

Why the other options are wrong:

Request a higher vCPU quota from the cloud provider, since the error means the account has reached its limit on how many instances it can run: Quota exhaustion is a different provider error; the question specifically reports insufficient instance capacity in the zone. Requesting more quota does not create unavailable physical capacity.

Pin older versions of the job's libraries, since an insufficient capacity error means a library conflict stopped the workers from joining: Library resolution occurs after compute has started, so library versions do not explain a provider instance-allocation failure. The termination reason points to infrastructure capacity.

Remove the cluster's init scripts, since an insufficient capacity error means a script used up the instance's resources during startup: Init scripts execute on provisioned compute during startup; they cannot cause the cloud provider to lack capacity to allocate the instance in the first place.

References:

1. Classic compute configuration best practices | Databricks on AWS:

<https://docs.databricks.com/aws/en/compute/cluster-config-best-practices>

Question 38

A finance close job must run at 18:00 on the last day of every month, whatever that month's length, whether or not its source tables changed that day. The schedule uses Quartz cron syntax.

Which expression fits?

A. 0 0 18 L * ?

B. 0 18 L * *

C. 0 0 18 31 * ?

D. 0 0 18 ? * L

Correct answer: A

A. 0 0 18 L * ?

Explanation

Correct answer:

0 0 18 L * ?

Why this is correct:

Quartz cron uses L in the day-of-month field to mean the last calendar day of the month, regardless of whether that month ends on the 28th, 30th, or 31st. The remaining fields set the time to 18:00 and leave day-of-week unspecified. A scheduled trigger runs by time, so it does not depend on table changes.

Why the other options are wrong:

0 0 18 31 * ?: Day 31 does not occur in every month, so this expression cannot schedule the close on each month's final date. It would miss shorter months.

0 0 18 ? * L: L in the day-of-week field means a last weekday occurrence, not the last day of the month. The question requires a calendar-month boundary.

0 18 L * *: This has too few fields for the stated Quartz format and places values in the wrong positions. It does not encode seconds, minutes, hour, and last day-of-month correctly.

References:

1. Job | Databricks API Reference: <https://docs.databricks.com/api/jobs/v2/job>

Question 39

A COPY INTO job has loaded a year of files from a volume into corp.bronze.events. The team now creates corp.bronze.events_v2 with a new schema and runs COPY INTO against it from the same path, without force.

What does the first load into events_v2 do?

- A. It loads the files in the path, because COPY INTO tracks the files it has loaded per target table, and events_v2 has not loaded any of them yet
- B. It fails, because a file that is already loaded into one table can be loaded into another table only after force is set to true for that run
- C. It loads nothing, because COPY INTO records each file as loaded for the whole source path, so any table that reads from that path now skips them
- D. It loads only the files added after the original table's last load, because the new table inherits that table's load history when it is created

Correct answer: A

A. It loads the files in the path, because COPY INTO tracks the files it has loaded per target table, and events_v2 has not loaded any of them yet

Explanation

Correct answer:

It loads the files in the path, because COPY INTO tracks the files it has loaded per target table, and events_v2 has not loaded any of them yet

Why this is correct:

COPY INTO keeps a record of files loaded into each target table, not a global consumed-file ledger for the source path. Since events_v2 is a new target with no prior load history, the files are eligible on its first COPY INTO even though another table previously loaded them. The different target schema does not inherit the first table's tracking state.

Why the other options are wrong:

It loads nothing, because COPY INTO records each file as loaded for the whole source path, so any table that reads from that path now skips them: File tracking is associated with the target table's load history, not every table that points at the same directory. A second target can ingest those files independently.

It fails, because a file that is already loaded into one table can be loaded into another table only after force is set to true for that run: force=true is used when you intentionally need to reload files already recorded for that same target. The new target has not loaded them before.

It loads only the files added after the original table's last load, because the new table inherits that table's load history when it is created: The load record belongs to the destination's transaction history. Since this destination is new, it has no record that these files were ingested elsewhere.

References:

1. COPY INTO | Databricks on AWS:

<https://docs.databricks.com/aws/en/sql/language-manual/delta-copy-into>

Question 40

Joins between silver customers and a region lookup miss many matches, although the values look identical. Profiling shows some state codes in silver carry leading or trailing spaces, such as ' NY' and 'CA '.

Which cleaning step fits?

- A. Apply upper to the state code when building silver, so the stored values all share one letter case and match the lookup table's codes exactly
- B. Apply coalesce to the state code when building silver, so the stored values replace their blank spaces with an empty string before the lookup join
- C. Apply trim to the state code when building silver, so the stored values drop the extra blank characters at either end and match the lookup's codes
- D. Cast the state code to VARCHAR(2) when building silver, so the stored values are cut down to two characters and match the lookup table's codes

Correct answer: C

C. Apply trim to the state code when building silver, so the stored values drop the extra blank characters at either end and match the lookup's codes

Explanation

Correct answer:

Apply trim to the state code when building silver, so the stored values drop the extra blank characters at either end and match the lookup's codes

Why this is correct:

trim removes leading and trailing spaces from a string, so ' NY' becomes 'NY' and 'CA ' becomes 'CA'. Cleaning the state code before joining makes values compare equal to the canonical lookup values without changing their case or truncating characters. It directly corrects the whitespace discrepancy found by profiling.

Why the other options are wrong:

Apply coalesce to the state code when building silver, so the stored values replace their blank spaces with an empty string before the lookup join: coalesce replaces NULL values, not spaces inside non-NULL strings. These codes contain actual characters plus surrounding blanks, so they remain unchanged.

Apply upper to the state code when building silver, so the stored values all share one letter case and match the lookup table's codes exactly: Uppercasing changes letter case but leaves leading and trailing spaces in place. It would not repair the discrepancy shown in the examples.

Cast the state code to VARCHAR(2) when building silver, so the stored values are cut down to two characters and match the lookup table's codes: Casting to a two-character type does not reliably remove leading spaces and can truncate values rather than normalize whitespace. Use a function designed to trim ends.

References:

1. trim function | Databricks on AWS:

<https://docs.databricks.com/aws/en/sql/language-manual/functions/trim>

Question 41

After a nightly load, the team wants the job to evaluate a saved SQL alert on the loaded table and notify its subscribers when the error count exceeds a threshold, without writing any notebook code.

Which task fits?

- A. An If/else task that depends on the load task, which runs the saved alert's query and then notifies the alert's subscribers on its true branch
- B. A SQL Alert task that depends on the load task, which runs the query behind that alert, checks its threshold, and then notifies its subscribers
- C. A dashboard task that depends on the load task, which refreshes the dashboard and then notifies its subscribers whenever a value crosses a limit
- D. A Run Job task that depends on the load task, which starts the saved alert's own schedule and then notifies its subscribers when the condition holds

Correct answer: B

B. A SQL Alert task that depends on the load task, which runs the query behind that alert, checks its threshold, and then notifies its subscribers

Explanation

Correct answer:

A SQL Alert task that depends on the load task, which runs the query behind that alert, checks its threshold, and then notifies its subscribers

Why this is correct:

A SQL Alert task evaluates an existing Databricks SQL alert as a step in a job. It runs the alert's query, checks its configured condition, and sends notifications to the configured subscribers, so the load can be a dependency and no notebook code is needed.

Why the other options are wrong:

A dashboard task that depends on the load task, which refreshes the dashboard and then notifies its subscribers whenever a value crosses a limit: Refreshing dashboard content does not evaluate an alert threshold or notify alert subscribers when the condition is met. Alerts are a separate SQL task type.

An If/else task that depends on the load task, which runs the saved alert's query and then notifies the alert's subscribers on its true branch: An If/else task branches on a condition configured for that job, but it does not run a saved alert or use its subscriber notifications. Use the task that directly evaluates the alert.

A Run Job task that depends on the load task, which starts the saved alert's own schedule and

then notifies its subscribers when the condition holds: This task starts another configured job, not an alert evaluation. The alert can be run directly as a dependent task without changing its independent schedule.

References:

1. **SQL alert task for jobs | Databricks on AWS:** <https://docs.databricks.com/aws/en/jobs/tasks/alert>

Question 42

A query filters a large Delta table with `WHERE date(event_ts) = '2026-09-01'`. The query profile shows almost no files pruned, although `event_ts` has file statistics and is a clustering key.

Which change should be tested first, then remeasured?

- A. Filter with `event_ts >= '2026-09-01' AND event_ts < '2026-09-02'`, so the predicate compares the raw column and files can be skipped
- B. Filter with `CAST(event_ts AS DATE) = '2026-09-01'`, so the predicate compares a date value directly and files can be skipped by their date
- C. Raise `spark.sql.shuffle.partitions` for the query, so the full scan of the table is spread across many more tasks and finishes sooner overall
- D. Run `ANALYZE TABLE` on the table first, so the planner learns the date range of each data file and can then prune files for the date filter

Correct answer: A

A. Filter with `event_ts >= '2026-09-01' AND event_ts < '2026-09-02'`, so the predicate compares the raw column and files can be skipped

Explanation

Correct answer:

Filter with `event_ts >= '2026-09-01' AND event_ts < '2026-09-02'`, so the predicate compares the raw column and files can be skipped

Why this is correct:

Functions applied to a filter column can make it harder for the optimizer to use the column's file statistics or clustering layout for data skipping. A half-open range on the raw `event_ts` value preserves a direct comparison that can be tested for file pruning, while covering all timestamps on the selected date without including the next day.

Why the other options are wrong:

Raise `spark.sql.shuffle.partitions` for the query, so the full scan of the table is spread across many more tasks and finishes sooner overall: More shuffle partitions affect downstream task parallelism, not which files qualify for scanning. The query profile points to ineffective file pruning as the first issue.

Filter with `CAST(event_ts AS DATE) = '2026-09-01'`, so the predicate compares a date value directly and files can be skipped by their date: Casting the column still wraps the stored value in an expression and may not enable direct statistics-based pruning. Compare the raw column to timestamp/date boundaries first.

Run ANALYZE TABLE on the table first, so the planner learns the date range of each data file and can then prune files for the date filter: ANALYZE TABLE gathers table-level statistics for planning, but it does not replace the per-file statistics used for data skipping or make the wrapped predicate sargable. The expression is the immediate issue to test.

References:

1. **Data skipping | Databricks on AWS:** <https://docs.databricks.com/aws/en/delta/data-skipping>

Question 43

The bundle variable catalog defaults to dev_cat, the prod target sets it to prod_cat, and the CI job exports BUNDLE_VAR_catalog=ci_cat before running databricks bundle deploy -t prod --var="catalog=hotfix_cat".

Which value does the deployment use?

- A. dev_cat, because the declared default value takes precedence over values given on the command line, environment variables, and target values
- B. prod_cat, because a target's own variable value takes precedence over values given on the command line, environment variables, and the default
- C. hotfix_cat, because a --var value given on the command line takes precedence over environment variables, target values, and the default value
- D. ci_cat, because a BUNDLE_VAR_ environment variable takes precedence over values given on the command line, target values, and the default value

Correct answer: C

C. hotfix_cat, because a --var value given on the command line takes precedence over environment variables, target values, and the default value

Explanation

Correct answer:

hotfix_cat, because a --var value given on the command line takes precedence over environment variables, target values, and the default value

Why this is correct:

Declarative Automation Bundles resolve custom variable values in this order: command-line --var, BUNDLE_VAR_ environment variable, override file, target mapping, then the top-level default. Because the deployment command supplies catalog=hotfix_cat, it takes precedence over both ci_cat and the prod target's prod_cat.

Why the other options are wrong:

prod_cat, because a target's own variable value takes precedence over values given on the command line, environment variables, and the default: Target values are lower priority than a command-line --var override. The target value is used only when no higher-priority value is supplied.

ci_cat, because a BUNDLE_VAR_ environment variable takes precedence over values given on the command line, target values, and the default value: The variable lookup stops at the first supplied value in the precedence chain. Since the command line provided one, the environment setting is never selected.

dev_cat, because the declared default value takes precedence over values given on the command line, environment variables, and target values: Defaults are fallbacks for deployments that provide no higher-priority value; here the explicit command-line assignment wins before resolution reaches the default.

References:

1. Substitutions and variables in Declarative Automation Bundles | Databricks on AWS:

<https://docs.databricks.com/aws/en/dev-tools/bundles/variables>

Question 44

Each row of a routes DataFrame holds an ordered array of stop names. The output needs one row per stop that also records the stop's position in the array, so the original order can be rebuilt later.

Which function fits?

- A. explode on the stops array, which returns one row per element and keeps a hidden index column that orders those rows by their original array position
- B. posexplode on the stops array, which returns one row per element, with a pos column holding its position and a col column holding its stop name
- C. explode_outer on the stops array, which returns one row per element, together with a pos column that holds the stop's position within the array
- D. flatten on the stops array, which returns one row per element, together with a pos column that holds the stop's position in the original array

Correct answer: B

B. posexplode on the stops array, which returns one row per element, with a pos column holding its position and a col column holding its stop name

Explanation

Correct answer:

posexplode on the stops array, which returns one row per element, with a pos column holding its position and a col column holding its stop name

Why this is correct:

posexplode expands an array into one row per element and emits both its zero-based position (pos) and value (col). Retaining that position makes it possible to reconstruct each route's original stop order after the rows are sorted or grouped. Ordinary explode returns values without the index.

Why the other options are wrong:

flatten on the stops array, which returns one row per element, together with a pos column that holds the stop's position in the original array: flatten combines nested arrays into a single array within each input row; it does not produce one output row per stop or provide element positions.

explode_outer on the stops array, which returns one row per element, together with a pos column that holds the stop's position within the array: explode_outer emits elements and preserves a row for NULL or empty arrays, but it has no position output. The requirement needs an ordinal index alongside every stop.

explode on the stops array, which returns one row per element and keeps a hidden index column

that orders those rows by their original array position: Without an explicit position output, row order is not guaranteed and the original array indexes are lost. Use a generator that returns positions with the values.

References:

1. **pyspark.sql.functions.posexplode | Apache Spark:** <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.functions.posexplode.html>

Question 45

A table already has a row filter for region access. Compliance adds a second rule that hides archived rows from non-auditors, and both rules must apply to every query on the table.

How should the second rule be added?

- A. Create a view with the archived rule over the table, since any view that selects from the table replaces the table's own row filter for its readers
- B. Set a second row filter on the table with SET ROW FILTER, since all of the row filters set on one table are combined with AND at query time
- C. Combine both conditions in one filter function and set it as the table's row filter, since a table can have only one row filter at a time
- D. Attach the archived rule as a column mask on the status column, since a mask that returns NULL for a row removes that row from query results

Correct answer: C

- C. Combine both conditions in one filter function and set it as the table's row filter, since a table can have only one row filter at a time

Explanation

Correct answer:

Combine both conditions in one filter function and set it as the table's row filter, since a table can have only one row filter at a time

Why this is correct:

A table has only one row-filter function that resolves at runtime for a given user and table. To enforce both rules on every query, put their logic together in a single predicate, for example by combining the existing region condition with the archived-row restriction using AND. A view would not globally replace the policy applied to the base table.

Why the other options are wrong:

Set a second row filter on the table with SET ROW FILTER, since all of the row filters set on one table are combined with AND at query time: The table cannot attach a second independent row filter and combine both automatically. The supported design is to incorporate the required logic in one filter function.

Create a view with the archived rule over the table, since any view that selects from the table replaces the table's own row filter for its readers: Queries through a view remain subject to the underlying table's row filter. A view also would not make the additional rule apply to every query against that table.

Attach the archived rule as a column mask on the status column, since a mask that returns NULL for a row removes that row from query results: Masking is applied to column values after the row is selected; it cannot enforce the requested row-level exclusion. The archived-row condition belongs in the filter predicate.

References:

1. Row filters and column masks | Databricks on AWS:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/filters-and-masks>

Question 46

A dashboard on a SQL warehouse is fast when one analyst uses it, but at month-end fifty analysts open it at once and their queries sit queued. Each query alone finishes quickly on the current cluster size.

Which change fits?

- A. Raise the warehouse's maximum cluster count, so additional clusters start when queries queue and the concurrent load is spread across all of them
- B. Switch the warehouse to the Classic type, so each analyst's session gets a dedicated cluster of its own and no query has to wait in the queue
- C. Lower the warehouse's auto stop time, so idle clusters shut down sooner and the capacity that they free up is handed over to the queued queries
- D. Raise the warehouse's cluster size, so each query runs on a larger cluster and the queue clears as every query then finishes a little faster

Correct answer: A

A. Raise the warehouse's maximum cluster count, so additional clusters start when queries queue and the concurrent load is spread across all of them

Explanation

Correct answer:

Raise the warehouse's maximum cluster count, so additional clusters start when queries queue and the concurrent load is spread across all of them

Why this is correct:

The individual queries finish quickly, but many concurrent requests wait for capacity. Increasing the warehouse's maximum cluster count lets it scale out additional clusters to serve queued work in parallel; changing the size of each cluster primarily changes per-query resources. The bottleneck described is concurrency capacity, not the speed of one query.

Why the other options are wrong:

Switch the warehouse to the Classic type, so each analyst's session gets a dedicated cluster of its own and no query has to wait in the queue: Queued short queries point to insufficient parallel serving capacity. Raising the maximum cluster count gives the warehouse room to scale out for concurrent requests.

Lower the warehouse's auto stop time, so idle clusters shut down sooner and the capacity that they free up is handed over to the queued queries: Idle shutdown only affects unused clusters after they have sat idle; it does not increase the active cluster ceiling that is holding queries in the queue.

Raise the warehouse's cluster size, so each query runs on a larger cluster and the queue clears as every query then finishes a little faster: A larger cluster can speed up an individual query, but the stem says each query already runs quickly and the issue is many simultaneous queued queries. More clusters address concurrency directly.

References:

1. SQL warehouse sizing, scaling, and queuing behavior | Databricks on AWS:

<https://docs.databricks.com/aws/en/compute/sql-warehouse/warehouse-behavior>

Question 47

An Auto Loader stream reads JSON with an explicitly supplied schema and no rescued data column. A new field appears in the source files. The stream keeps running, but the field never shows up anywhere in the target table.

What explains this behavior?

- A. With a supplied schema the default mode is failOnNewColumns, which drops new fields quietly and fails the stream only when a column type changes
- B. With a supplied schema the default mode is rescue, which moves the new fields into the checkpoint instead of into a rescued data column of the table
- C. With a supplied schema the default mode is none, which ignores new columns and rescues nothing unless a rescued data column has been configured
- D. With a supplied schema the default mode is addNewColumns, which adds the new fields to the table only after its next full refresh has completed

Correct answer: C

C. With a supplied schema the default mode is none, which ignores new columns and rescues nothing unless a rescued data column has been configured

Explanation

Correct answer:

With a supplied schema the default mode is none, which ignores new columns and rescues nothing unless a rescued data column has been configured

Why this is correct:

With an explicitly supplied Auto Loader schema, the default schema evolution mode is none. New columns are ignored, and they are not rescued unless a rescued-data column has been configured. That explains why the stream remains healthy but the unexpected field is absent both from the table schema and any rescue output.

Why the other options are wrong:

With a supplied schema the default mode is rescue, which moves the new fields into the checkpoint instead of into a rescued data column of the table: The rescue behavior requires selecting that mode; with a supplied schema, it is not the default. Rescued fields belong in a configured rescued-data column, not the checkpoint.

With a supplied schema the default mode is failOnNewColumns, which drops new fields quietly and fails the stream only when a column type changes: failOnNewColumns stops the stream when an unexpected column appears; it does not silently ignore the field while continuing. That conflicts with the observed successful run.

With a supplied schema the default mode is addNewColumns, which adds the new fields to the table only after its next full refresh has completed: Waiting for a refresh will not change the configured evolution mode. Set an appropriate schema-evolution mode if new columns should enter the table schema.

References:

1. Configure schema inference and evolution in Auto Loader | Databricks on AWS:

<https://docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/schema>

Question 48

An external location holds a large Parquet directory that no process writes to anymore. The team wants Delta features on this data without rewriting or copying any of the existing files.

Which approach fits?

- A. Run `CONVERT TO DELTA` on the directory, which writes a transaction log over the existing Parquet files, and then register the table
- B. Run `OPTIMIZE` on the Parquet directory, which rewrites its files into Delta format and adds a transaction log under the same path for the new table
- C. Run `CREATE TABLE AS SELECT` from the Parquet files, which writes a Delta table from them in place at the same storage path and reuses the existing files
- D. Create an external table over the directory `USING PARQUET`, which gives the files Delta features such as time travel without rewriting them

Correct answer: A

A. Run `CONVERT TO DELTA` on the directory, which writes a transaction log over the existing Parquet files, and then register the table

Explanation

Correct answer:

Run `CONVERT TO DELTA` on the directory, which writes a transaction log over the existing Parquet files, and then register the table

Why this is correct:

`CONVERT TO DELTA` adds Delta transaction-log metadata for existing Parquet data files and converts the directory into a Delta table without rewriting or copying those files. After conversion, registering the table makes it addressable through the catalog. The directory must remain quiescent during conversion so the inventory and log stay consistent.

Why the other options are wrong:

Run `OPTIMIZE` on the Parquet directory, which rewrites its files into Delta format and adds a transaction log under the same path for the new table: `OPTIMIZE` is a Delta table maintenance operation and does not convert a Parquet directory into Delta by itself. Rewriting files is also unnecessary for this metadata conversion.

Create an external table over the directory `USING PARQUET`, which gives the files Delta features such as time travel without rewriting them: Registering the directory as a Parquet table provides ordinary file access but not Delta transaction-log features such as versioned transactions. A Parquet registration is not a format conversion.

Run CREATE TABLE AS SELECT from the Parquet files, which writes a Delta table from them in place at the same storage path and reuses the existing files: CTAS writes query results as a new table rather than reusing the source Parquet files in place. It would create a copy and does not establish Delta over the original data path as requested.

References:

1. Convert to Delta Lake | Databricks on AWS:

<https://docs.databricks.com/aws/en/ingestion/data-migration/convert-to-delta>

Question 49

A notebook task appends a day's orders to a Delta table and sometimes fails partway on a network error. The team wants automatic task retries, but a retried attempt must never duplicate the day's rows.

How should the retries be set up?

- A. Enable a bounded number of retries with a long minimum retry interval, so the earlier attempt's appended rows can finish committing first
- B. Enable retries with retry on timeout turned on, so any failed attempt's appended rows are rolled back before the next attempt of the task starts
- C. Enable unlimited retries on the task, so the append is repeated until one attempt succeeds and writes all of the day's rows to the table in full
- D. Make the write idempotent, for example a MERGE on order_id or a replaceWhere for that day, and then enable a bounded number of task retries

Correct answer: D

D. Make the write idempotent, for example a MERGE on order_id or a replaceWhere for that day, and then enable a bounded number of task retries

Explanation

Correct answer:

Make the write idempotent, for example a MERGE on order_id or a replaceWhere for that day, and then enable a bounded number of task retries

Why this is correct:

Task retries rerun work after failure, so the write must be safe to apply more than once. An idempotent MERGE keyed by order ID or a replaceWhere restricted to that business date makes a repeated attempt converge on the same table state rather than append duplicates. After making the write idempotent, enable a finite retry count.

Why the other options are wrong:

Enable a bounded number of retries with a long minimum retry interval, so the earlier attempt's appended rows can finish committing first: Elapsed time cannot reverse a partially committed write. Make the operation converge to one day's intended state before enabling retries.

Enable unlimited retries on the task, so the append is repeated until one attempt succeeds and writes all of the day's rows to the table in full: A retry policy cannot correct duplicate-producing semantics and may repeatedly execute them. Bound retries after making each attempt safe to repeat.

Enable retries with retry on timeout turned on, so any failed attempt's appended rows are rolled

back before the next attempt of the task starts: Retry-on-timeout controls whether a timed-out run is retried; it does not roll back a partial external or Delta append. Idempotency is the mechanism that makes retries safe.

References:

1. Use foreachBatch to write to arbitrary data sinks | Databricks on AWS:

<https://docs.databricks.com/aws/en/structured-streaming/foreach>

Question 50

A Lakeflow Spark Declarative Pipelines SQL file must ingest JSON files that keep landing in a volume, processing each new file once and appending its records to a bronze table.

Which definition fits?

A. `CREATE OR REFRESH MATERIALIZED VIEW bronze AS SELECT * FROM read_files('/Volumes/corp/raw/events', format => 'json')`

B. `CREATE OR REFRESH STREAMING TABLE bronze AS SELECT * FROM read_files('/Volumes/corp/raw/events', format => 'json')`

C. `CREATE OR REFRESH STREAMING TABLE bronze AS SELECT * FROM STREAM read_files('/Volumes/corp/raw/events', format => 'json')`

D. `CREATE VIEW bronze AS SELECT * FROM STREAM read_files('/Volumes/corp/raw/events', format => 'json')`

Correct answer: C

C. `CREATE OR REFRESH STREAMING TABLE bronze AS SELECT * FROM STREAM read_files('/Volumes/corp/raw/events', format => 'json')`

Explanation

Correct answer:

`CREATE OR REFRESH STREAMING TABLE bronze AS SELECT * FROM STREAM read_files('/Volumes/corp/raw/events', format => 'json')`

Why this is correct:

In a Lakeflow Spark Declarative Pipelines SQL definition, `STREAM read_files(...)` marks the file source as streaming input, and `CREATE OR REFRESH STREAMING TABLE` incrementally processes new data. That pairing gives the bronze table append-oriented streaming semantics and tracks files through the pipeline's checkpointing. The non-streaming alternatives do not express this source behavior.

Why the other options are wrong:

CREATE OR REFRESH STREAMING TABLE bronze AS SELECT * FROM read_files('/Volumes/corp/raw/events', format => 'json'); Although this creates a streaming table, the input is a batch `read_files` call without `STREAM`, so it does not declare incremental streaming consumption of arriving files.

CREATE OR REFRESH MATERIALIZED VIEW bronze AS SELECT * FROM

read_files('/Volumes/corp/raw/events', format => 'json'): A materialized view is refreshed from a query result rather than acting as the append-only streaming-table target requested. It is not the intended once-per-new-file ingestion object.

CREATE VIEW bronze AS SELECT * FROM STREAM read_files('/Volumes/corp/raw/events', format => 'json'): A standard view is a query definition, not a managed incremental table. Adding STREAM at the source does not make CREATE VIEW a streaming table declaration.

References:

1. Load data in pipelines | Databricks on AWS: <https://docs.databricks.com/aws/en/ldp/load>

Question 51

A schema holds dozens of Unity Catalog managed tables, and more are added each month. The team wants predictive optimization for all current and future tables in that schema, while the rest of the catalog stays as it is.

Which statement fits?

- A. ALTER TABLE with ENABLE PREDICTIVE OPTIMIZATION on each current table, which also covers the tables that are later created in that schema
- B. A nightly OPTIMIZE on each table in the schema, which turns on predictive optimization for every table that the schema contains
- C. ALTER SCHEMA with ENABLE PREDICTIVE OPTIMIZATION, which covers the schema's tables, as each table inherits that setting by default
- D. ALTER CATALOG with ENABLE PREDICTIVE OPTIMIZATION, which covers the chosen schema's tables and leaves the catalog's other schemas as they are

Correct answer: C

C. ALTER SCHEMA with ENABLE PREDICTIVE OPTIMIZATION, which covers the schema's tables, as each table inherits that setting by default

Explanation

Correct answer:

ALTER SCHEMA with ENABLE PREDICTIVE OPTIMIZATION, which covers the schema's tables, as each table inherits that setting by default

Why this is correct:

Predictive optimization settings inherit from parent Unity Catalog objects. Enabling it on the schema covers tables in that schema that inherit the setting, including future tables, while leaving sibling schemas under their own catalog setting. A schema-level change therefore matches the requested scope.

Why the other options are wrong:

A nightly OPTIMIZE on each table in the schema, which turns on predictive optimization for every table that the schema contains: Running OPTIMIZE is a manual maintenance operation, not a persistent predictive-optimization setting. It also must be scheduled and repeated for new tables.

ALTER TABLE with ENABLE PREDICTIVE OPTIMIZATION on each current table, which also covers the tables that are later created in that schema: Enabling the feature table by table affects only those tables and does not set the default for future schema members. The schema is the needed inheritance scope.

ALTER CATALOG with ENABLE PREDICTIVE OPTIMIZATION, which covers the chosen schema's

tables and leaves the catalog's other schemas as they are: Catalog scope includes its other schemas, so it is broader than requested. The schema-level setting isolates this change to the target schema.

References:

1. Predictive optimization for Unity Catalog managed tables | Databricks on Google Cloud:
<https://docs.databricks.com/gcp/en/optimizations/predictive-optimization>

Question 52

An engineer joins two DataFrames with `a.join(b, a.id == b.id)`. Later, selecting the `id` column raises an ambiguous reference error, and the output must contain a single `id` column without a renaming step.

Which change fixes this?

- A. Join with `a.crossJoin(b).filter(a.id == b.id)`, which matches the same rows and merges the two `id` columns into one column in the output
- B. Join with `a.join(b, ["id"])`, which matches rows on the shared column name and keeps a single `id` column in the joined DataFrame's final output
- C. Join with `a.join(b, a.id == b.id, "left")`, which matches the same rows and keeps only the left DataFrame's `id` column in the joined output
- D. Join with `a.join(b, a.id == b.id).distinct()`, which matches the same rows and then removes the duplicate `id` column from the joined output

Correct answer: B

B. Join with `a.join(b, ["id"])`, which matches rows on the shared column name and keeps a single `id` column in the joined DataFrame's final output

Explanation

Correct answer:

Join with `a.join(b, ["id"])`, which matches rows on the shared column name and keeps a single `id` column in the joined DataFrame's final output

Why this is correct:

When a DataFrame join uses a shared column name such as `"id"`, Spark treats the key as one join column in the output. Joining with an explicit boolean expression (`a.id == b.id`) retains both input key columns, which can make a later unqualified `id` ambiguous. The name-based join avoids that duplicate key without a rename step.

Why the other options are wrong:

Join with `a.join(b, a.id == b.id, "left")`, which matches the same rows and keeps only the left DataFrame's `id` column in the joined output: Changing the join type to left affects which unmatched rows are retained, not whether an expression join preserves both key columns. The ambiguity remains.

Join with `a.crossJoin(b).filter(a.id == b.id)`, which matches the same rows and merges the two `id` columns into one column in the output: A cross join creates every pair and the filter only selects matching pairs; it does not merge duplicate key columns into one. It also performs unnecessary Cartesian work.

Join with `a.join(b, a.id == b.id).distinct()`, which matches the same rows and then removes the duplicate id column from the joined output: Deduplicating records cannot resolve two schema fields that share the same name. Rename or select the key explicitly if an expression join is used; here the name-based join avoids that ambiguity.

References:

1. pyspark.sql.DataFrame.join | Apache Spark: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.join.html>

Question 53

A new Auto Loader stream must first process a backlog of 400,000 files, and its first micro-batch runs out of memory. The team wants smaller, steadier micro-batches without changing the cluster or skipping any existing file.

Which change fits?

- A. Set a processingTime trigger of ten minutes, so each micro-batch waits longer between runs and so takes a smaller share of the backlog each time
- B. Set cloudFiles.maxBytesPerTrigger or cloudFiles.maxFilesPerTrigger, so each micro-batch takes only a bounded share of the existing backlog
- C. Set cloudFiles.includeExistingFiles to false, so the backlog is left out of the first micro-batch and only the newly arriving files are processed
- D. Set cloudFiles.useNotifications to true, so the backlog arrives through storage notifications in small batches instead of one large directory listing

Correct answer: B

B. Set cloudFiles.maxBytesPerTrigger or cloudFiles.maxFilesPerTrigger, so each micro-batch takes only a bounded share of the existing backlog

Explanation

Correct answer:

Set cloudFiles.maxBytesPerTrigger or cloudFiles.maxFilesPerTrigger, so each micro-batch takes only a bounded share of the existing backlog

Why this is correct:

Auto Loader's cloudFiles.maxBytesPerTrigger and cloudFiles.maxFilesPerTrigger cap the amount of backlog admitted to each micro-batch. That reduces the size of each batch while still processing the existing files over successive batches. The cap should be tuned to the available memory and desired progress rate.

Why the other options are wrong:

Set cloudFiles.includeExistingFiles to false, so the backlog is left out of the first micro-batch and only the newly arriving files are processed: Disabling existing files excludes the 400,000-file backlog entirely, contradicting the requirement to process every existing file. It is not a batch-sizing control.

Set cloudFiles.useNotifications to true, so the backlog arrives through storage notifications in small batches instead of one large directory listing: Notifications change file discovery mechanics but do not guarantee smaller batches or remove the need to process the backlog. Trigger limits directly bound batch input.

Set a processingTime trigger of ten minutes, so each micro-batch waits longer between runs and so takes a smaller share of the backlog each time: Processing-time triggers control when batches run, not how much input each batch reads. Waiting longer can leave the same oversized backlog available to the next batch.

References:

1. Spark API options reference | Databricks on AWS:

<https://docs.databricks.com/aws/en/spark/api-options>

Question 54

A niche SaaS tool exposes a paginated REST API. No Lakeflow Connect managed connector or Partner Connect partner supports it, and the team needs its records in a Delta table every night.

Which approach fits?

- A. A COPY INTO job pointed at the API's base URL, which loads each page of records once and skips the pages that it loaded on earlier runs
- B. A notebook that calls the API with a Python HTTP client, follows its paging, and writes the records to Delta, run each night as a scheduled job
- C. Lakehouse Federation with a connection to the API's base URL, which exposes the tool's records as a foreign catalog that analysts can query
- D. Auto Loader pointed at the API's base URL, which discovers each new page of records in the same way that it discovers new files landing in cloud storage

Correct answer: B

B. A notebook that calls the API with a Python HTTP client, follows its paging, and writes the records to Delta, run each night as a scheduled job

Explanation

Correct answer:

A notebook that calls the API with a Python HTTP client, follows its paging, and writes the records to Delta, run each night as a scheduled job

Why this is correct:

When no managed or partner connector supports a paginated API, a notebook can implement the API's authentication, pagination, and response handling, then write the records to Delta. Scheduling that notebook as a job supplies the requested nightly cadence. This is a custom integration, unlike file discovery or SQL loading from storage.

Why the other options are wrong:

Auto Loader pointed at the API's base URL, which discovers each new page of records in the same way that it discovers new files landing in cloud storage: File discovery in cloud storage does not issue paginated HTTP requests to an API endpoint. An API's pages are not files arriving in an object-store directory.

Lakehouse Federation with a connection to the API's base URL, which exposes the tool's records as a foreign catalog that analysts can query: Lakehouse Federation supports specific database sources and query protocols, not an arbitrary REST endpoint as a foreign catalog. The team needs a client that understands the SaaS API.

A COPY INTO job pointed at the API's base URL, which loads each page of records once and skips the pages that it loaded on earlier runs: COPY INTO loads supported files from a storage path; it does not call REST APIs or follow page tokens. API pagination requires application logic.

References:

1. Lakeflow Connect connector concepts | Databricks on AWS:

<https://docs.databricks.com/aws/en/ingestion/lakeflow-connect>

Question 55

A Salesforce admin adds a new custom field to an object that a Lakeflow Connect pipeline already ingests. The pipeline selects all columns of that object, and nobody has changed its configuration since it was created.

What happens on the next pipeline update?

- A.** The new column is added to the destination table automatically on that update, with NULL values in rows ingested before the field existed
- B.** The new column is ignored until someone edits the pipeline to select it, as the column selection is fixed when a pipeline is first created
- C.** The new column's values land in a rescued data column of the destination table, as managed connectors never change a destination's schema
- D.** The update fails on the schema change, and the destination table then needs a full refresh before the new field can be ingested into it at all

Correct answer: A

A. The new column is added to the destination table automatically on that update, with NULL values in rows ingested before the field existed

Explanation

Correct answer:

The new column is added to the destination table automatically on that update, with NULL values in rows ingested before the field existed

Why this is correct:

Lakeflow Connect's Salesforce connector selects all object columns by default and evolves the destination schema as new fields appear. The new field is added on the update, while rows ingested before that field existed have no source value and therefore show NULL. No pipeline edit is needed for this default selection behavior.

Why the other options are wrong:

The new column's values land in a rescued data column of the destination table, as managed connectors never change a destination's schema: Schema rescue applies to Auto Loader's configured parsing behavior. This is a managed Salesforce connector, whose documented default is to ingest new columns automatically.

The update fails on the schema change, and the destination table then needs a full refresh before the new field can be ingested into it at all: The described default behavior is schema evolution, not a failure requiring a full refresh. Previously ingested rows can remain NULL while subsequent records include the new field.

The new column is ignored until someone edits the pipeline to select it, as the column selection is fixed when a pipeline is first created: The connector's default all-column selection is not frozen at pipeline creation. A custom selection would change that, but the stem says nobody changed the default configuration.

References:

1. Managed connector FAQs | Databricks on AWS:

<https://docs.databricks.com/aws/en/ingestion/lakeflow-connect/faq>